

ĐẠI HỌC CẦN THƠ
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA KHOA HỌC MÁY TÍNH

-----♣-----



NIÊN LUẬN CƠ SỞ NGÀNH
KHOA HỌC MÁY TÍNH

**Xây dựng Chatbot hỗ trợ tra cứu
quy định của Đại học Cần Thơ sử dụng kỹ thuật
RAG và Hybrid Search (FAISS + BM25)**

GIẢNG VIÊN HƯỚNG DẪN: SINH VIÊN THỰC HIỆN:
TS. Trần Việt Châu Phạm Duy Anh - MSSV: B2308346
Khoa học Máy tính - Khóa 49

Tháng 12/2025

ĐẠI HỌC CẦN THƠ
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA KHOA HỌC MÁY TÍNH

-----❧-----



NIÊN LUẬN CƠ SỞ NGÀNH
KHOA HỌC MÁY TÍNH

**Xây dựng Chatbot hỗ trợ tra cứu
quy định của Đại học Cần Thơ sử dụng kỹ thuật
RAG và Hybrid Search (FAISS + BM25)**

GIẢNG VIÊN HƯỚNG DẪN: SINH VIÊN THỰC HIỆN:

TS. Trần Việt Châu

Phạm Duy Anh - MSSV: B2308346

Khoa học Máy tính - Khóa 49

Tháng 12/2025

This image shows a full page of white paper with horizontal dashed lines. The lines are evenly spaced and run across the entire width of the page, providing a guide for handwriting practice. There are no margins, text, or other markings on the paper.

Giảng viên ký tên

PHẦN MỞ ĐẦU

Lời cảm ơn

Em xin gửi lời cảm ơn sâu sắc nhất đến với thầy – TS. Trần Việt Châu, người đã hướng dẫn và chỉ bảo cũng như định hướng em, góp ý cho em trong suốt quá trình thực hiện niên luận. Những góp ý từ kiến thức và kinh nghiệm quý báu của thầy đã truyền đạt đến em, đã giúp em rút ngắn được rất nhiều thời gian để hoàn thành niên luận này.

Em cũng xin cảm ơn các quý thầy cô của Trường đã giúp em trang bị các nền tảng kiến thức vững chắc để có thể sử dụng vào đề tài.

Do hạn chế về mặt kiến thức, thời gian và kinh nghiệm, đề tài không tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý của thầy cô để đề tài hoàn thiện hơn sau này.

Em xin chân thành cảm ơn!

Tóm tắt đề tài

Tên đề tài: Xây dựng hệ thống chatbot tự vấn quy định sử dụng Retrieval-Augmented Generation và mô hình ngôn ngữ lớn

Sinh viên thực hiện: Phạm Duy Anh - **MSSV:** B2308346- **Lớp:** DI23Z6A1

Giảng viên hướng dẫn: Ts. Trần Việt Châu

Tóm tắt đề tài:

Việc tra cứu quy chế sinh viên tại Trường Đại học Cần Thơ hiện chủ yếu thông qua tra cứu thủ công, gây tốn thời gian và không hiệu quả. Niên luận này đề xuất xây dựng hệ thống chatbot tự động sử dụng công nghệ Retrieval-Augmented Generation (RAG) và mô hình ngôn ngữ lớn Llama 3.1 8B chạy local.

Hệ thống được thiết kế theo kiến trúc ba tầng với pipeline RAG hoàn chỉnh: thu thập và tiền xử lý 14 văn bản quy chế CTU, chunking thành ~183chunks, embedding bằng vietnamese-sbert, lưu vào FAISS index và BM25 index. Khi nhận câu hỏi, hệ thống tìm kiếm song song bằng hybrid search (BM25 + FAISS), fusion bằng Reciprocal Rank Fusion, sau đó truyền context vào LLM Llama 3.3 để sinh câu trả lời streaming qua Server-Sent Events.

Kết quả đánh giá trên 50 câu hỏi thực tế cho thấy hệ thống đạt **84%** (42/50 câu đúng), độ tương đồng trung bình **81.2% accuracy** thời gian phản hồi trung bình 33.41 giây. Ưu điểm nổi bật: bảo mật cao (chạy local), chi phí thấp (miễn phí API), kiểm soát hoàn toàn, xử lý tiếng Việt tốt. Hạn chế: thời gian phản hồi chậm hơn cloud chatbot, 16% câu trả lời chưa chính xác do LLM hallucination.

Hướng phát triển bao gồm: sử dụng GPU và caching để tăng tốc, fine-tune LLM trên dữ liệu CTU, xây dựng dashboard quản trị, phát triển mobile app, và mở rộng sang multi-domain. Đề tài đã chứng minh tính khả thi của chatbot RAG local cho tổ chức giáo dục nhỏ, có thể triển khai thực tế và nhân rộng cho các trường khác.

Mục Lục

PHẦN MỞ ĐẦU	1
Lời cảm ơn	1
Tóm tắt đề tài.....	2
Mục Lục	3
Danh mục hình	7
Danh mục bảng.....	8
PHẦN NỘI DUNG	9
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI.....	9
1.1. Lý do chọn đề tài.....	9
1.2. Mục tiêu.....	9
1.3. Các chức năng chính	9
1.3.1. Phản hồi người dùng.....	9
1.3.2. Đăng ký, đăng nhập và chế độ “khách”	9
1.3.3. Lưu lịch sử hội thoại	10
1.4. Công nghệ sử dụng.....	10
1.5. Kết quả đạt được	10
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	11
2.1. Trí tuệ nhân tạo và chatbot.....	11
2.1.1. Chatbot là gì?	11
2.1.2. Ứng dụng của chatbot	11
2.2. Mô hình ngôn ngữ lớn (LLM).....	11
2.3. RAG (Retrieval-Augmented Generation).....	12
2.4. Tìm kiếm thông tin.....	13
2.4.1. Vector Embedding	13
2.4.2. FAISS (Tìm kiếm theo ngữ nghĩa).....	13
2.4.3. BM25 (Tìm kiếm theo từ khóa)	13
2.4.4. Hybrid Search (tìm kiếm lai / tìm kiếm kết hợp)	13
2.5. Các công nghệ sử dụng	13
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....	14
3.1. Yêu cầu hệ thống (System Requirements)	14
3.1.1. Yêu cầu chức năng (Functional Requirements)	14
3.1.2. Yêu cầu phi chức năng (Non-Functional Requirements).....	14

3.1.3.	Yêu cầu phần cứng và phần mềm	14
a.	Yêu cầu phần cứng (Hardware Requirements).....	14
b.	Yêu cầu phần mềm (Software Requirements)	15
c.	Môi trường phát triển hệ thống	15
3.2.	Kiến trúc hệ thống	15
3.2.1.	Tổng quan	15
3.2.2.	Sơ đồ kiến trúc hệ thống	16
3.2.3.	Luồng hoạt động của hệ thống.....	17
3.2.4.	Ưu điểm kiến trúc	20
3.3.	Thiết kế cơ sở dữ liệu (Database).....	22
3.3.1.	Sơ đồ thực thể - quan hệ (ERD).....	22
3.3.2.	Mô tả các bảng chính	23
3.3.3.	Cơ sở dữ liệu vector (FAISS Index)	25
3.3.4.	Cơ sở dữ liệu ngữ nghĩa (BM25 Index)	26
3.4.	Thiết kế hệ thống tìm kiếm.....	27
3.4.1.	Tổng quan Hybrid Search	27
3.4.2.	BM25 Index	27
3.4.3.	FAISS Vector Database	27
3.4.4.	Chiến lược kết hợp (RRF).....	28
3.5.	Thiết kế API	28
3.5.1.	Cấu trúc tổng thể.....	28
3.5.2.	Danh sách API chính	28
3.5.3.	Luồng xử lý API chính (/api/ask)	29
3.5.4.	Streaming Response	30
3.5.5.	Ưu điểm thiết kế API	30
3.6.	Thiết kế giao diện.....	30
3.6.1.	Nguyên tắc thiết kế	30
3.6.2.	Giao diện chính	30
a.	Trang đăng ký.....	30
b.	Trang đăng nhập	32
c.	Trang Chat	32
CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG		34
4.1.	Môi trường phát triển	34
4.1.1.	Phần cứng.....	34
4.1.2.	Phần mềm.....	34
4.1.3.	Các thư viện chính	34

4.2.	Chuẩn bị dữ liệu	34
4.2.1.	Giai đoạn thu thập văn bản	34
4.2.2.	Giai đoạn tiền xử lý văn bản	35
4.2.3.	Chunking.....	35
4.2.4.	Import vào MySQL.....	38
4.3.	Xây dựng hệ thống tìm kiếm.....	38
4.3.1.	Tạo FAISS Sematic Index.....	38
4.3.2.	Xây dựng BM25 Index	38
4.3.3.	Xây dựng tham số tối ưu cho tìm kiếm.....	39
4.4.	Xây dựng backend (FastAPI).....	39
4.4.1.	Cấu trúc dự án	39
4.4.2.	Triển khai RAG pipeline.....	40
4.4.3.	Tích hợp LLM (Ollama)	40
4.5.	Xây dựng Frontend.....	40
4.5.1.	Công nghệ	40
4.5.2.	Cấu trúc thư mục.....	40
4.5.3.	Xử lý streaming.....	41
4.6.	Kiểm thử hệ thống.....	42
4.6.1.	Kiểm thử chức năng.....	42
4.6.2.	Kiểm thử đánh giá.....	42
4.6.3.	Kiểm thử phi chức năng.....	43
4.7.	Triển khai (Deployment).....	43
4.7.1.	Khởi động Backend	43
4.7.2.	Khởi động Frontend	43
4.7.3.	Khởi động Ollama.....	43
4.7.4.	Kiểm tra hoạt động.....	43
4.7.5.	Test API	43
CHƯƠNG 5. ĐÁNH GIÁ VÀ KẾT QUẢ		44
5.1.	Phương pháp đánh giá.....	44
5.1.1.	Bộ dữ liệu đánh giá	44
5.1.2.	Tiêu chí đánh giá.....	44
5.1.3.	Quy trình đánh giá	45
5.2.	Kết quả đánh giá.....	45
5.2.1.	Kết quả tổng quan	45
5.2.2.	Phân tích chi tiết.....	46
5.3.	So sánh với các giải pháp khác.....	47

5.4.	Phân tích ưu điểm.....	48
5.5.	Phân tích hạn chế.....	48
5.5.1.	Về hiệu năng	48
5.5.2.	Về độ chính xác	48
5.5.3.	Về dữ liệu.....	48
5.5.4.	Về giao diện	49
PHẦN KẾT		50
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN		50
6.1.	Kết luận	50
6.1.1.	Tổng kết công việc đã thực hiện	50
6.1.2.	Kết quả đạt được	50
6.1.3.	Đóng góp của đề tài	51
6.1.4.	Ý nghĩa thực tiễn.....	51
6.2.	Hướng phát triển.....	51
6.2.1.	Cải thiện tập dữ liệu	51
6.2.2.	Cải thiện độ chính xác.....	51
6.2.3.	Tối ưu hiệu năng	52
6.2.4.	Mở rộng chức năng	52
Lời kết.....		52
TÀI LIỆU THAM KHẢO		53
Tiếng Anh		53
Tiếng Việt		53
Tài liệu kỹ thuật và Documentation		53
Dataset và Pre-trained Models		54
Quy chế và Văn bản pháp lý CTU		54
PHỤ LỤC		54
Hướng dẫn cài đặt và source code dự án.....		54

Danh mục hình

Hình 1-1 Giao diện bắt đầu của trang web.....	10
Hình 3-1 Kiến trúc tổng quan của hệ thống.....	16
Hình 3-2 Sơ đồ luồng xử lý câu hỏi.....	17
Hình 3-3 System_prompt cho LLM.....	19
Hình 3-4 Prompt template cho LLM.....	19
Hình 3-5 Sơ đồ thực thể - quan hệ của hệ thống.....	22
Hình 3-6 Luồng xử lý API /api/ask.....	29
Hình 3-7 Giao diện trang đăng ký.....	31
Hình 3-8 Giao diện trang đăng ký có form validation.....	31
Hình 3-9 Giao diện trang đăng nhập.....	32
Hình 3-10 Giao diện trang chủ người dùng khi chưa có đoạn chat.....	33
Hình 3-11 Giao diện trang chủ khi đã có lịch sử chat.....	33
Hình 4-1 Một đoạn văn bản sau khi chuẩn hóa.....	35
Hình 4-2 Sơ đồ phân bổ chunk theo từng điều trước khi cắt hợp lý.....	36
Hình 4-3 Sự phân bổ chunks theo từng file.....	36
Hình 4-4 Biểu đồ phân loại chunks theo kích thước.....	37
Hình 4-5 Đoạn chương trình về độ ưu tiên khi chunking.....	38
Hình 4-6 So sánh độ chính xác theo từng tham số.....	39
Hình 4-7 Xử lý hiển thị streaming response.....	41
Hình 5-1 Kết quả đạt được của dự án.....	46
Hình 5-2 Hiệu suất dựa vào nguồn dữ liệu.....	47

Danh mục bảng

Bảng 3-1 Yêu cầu phần cứng	14
Bảng 3-2 Yêu cầu phần mềm	15
Bảng 3-3 Bảng mô tả chi tiết table documents	23
Bảng 3-4 Bảng mô tả chi tiết table chapters	23
Bảng 3-5 Bảng mô tả chi tiết table articles	23
Bảng 3-6 Bảng mô tả chi tiết table clauses	24
Bảng 3-7 Bảng mô tả chi tiết table users	24
Bảng 3-8 Bảng mô tả chi tiết table conversations.....	24
Bảng 3-9 Bảng mô tả chi tiết table messages	25
Bảng 3-10 Bảng danh sách các API chính	28
Bảng 5-1 Bảng kết quả tổng quan	45
Bảng 5-2 Bảng so sánh với các giải pháp khác.....	47

PHẦN NỘI DUNG

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Lý do chọn đề tài

Lý do chọn đề tài khá đơn giản. Vì chính tôi, người thực hiện đề tài cảm thấy khá khó chịu khi gặp phải quá nhiều câu hỏi trùng lặp về các quy định hay nội quy của trường từ các hội nhóm sinh viên của trường trên mạng xã hội – thứ nằm trong các văn bản và quy định đã được công bố, ban hành công khai trên các nền tảng. Hơn thế nữa, các văn bản, nghị định khá dài và hàn lâm càng khiến sinh viên “lười” đọc. Đó là những lý do làm cho việc có chatbot để tự động tìm kiếm và sinh câu trả lời dựa trên các văn bản, nội quy của trường là cần thiết.

1.2. Mục tiêu

Mục tiêu của đề tài hướng đến việc xây dựng một chatbot có khả năng trả lời các câu hỏi về quy chế, quy định và nghị định của Đại học Cần Thơ.

Cùng với mục tiêu của học phần, đề tài có mục tiêu ứng dụng các mô hình học sâu như LLM (Large Language Model), FAISS và BM25 cùng các kỹ thuật RAG (Retrieval–Agument–Generation) và Hybrid (Tìm kiếm lai) vào bài toán thực tế - thứ không thể chỉ học bằng lý thuyết.

Ngoài ra, đề tài của niên luận cũng hướng đến một giao diện tối giản nhất có thể để có thể dễ sử dụng, tránh làm rối và phức tạp hóa vấn đề.

1.3. Các chức năng chính

1.3.1. Phản hồi người dùng

Khi người dùng hỏi, hệ thống sẽ tiến hành chuẩn hóa dữ liệu câu hỏi và tiến hành tìm kiếm trong cơ sở dữ liệu vector dựa vào ngữ nghĩa và độ lặp của “keyword” được trích xuất từ câu hỏi, sau đó trả về một chuỗi các thông tin tốt nhất được tìm thấy và gửi đến mô hình ngôn ngữ lớn chạy trên local để tiến hành sinh ngôn ngữ tự nhiên để trả lời người dùng phù hợp ngữ cảnh.

1.3.2. Đăng ký, đăng nhập và chế độ “khách”

Hệ thống cho phép người dùng khách hỏi và tìm kiếm thông tin mà không cần đăng nhập. Khi đó, lịch sử hội thoại – thứ được dùng làm ngữ cảnh trò chuyện sẽ được lưu vào local storage trên trình duyệt web.

Đối lập với chế độ khách, người dùng có thể đăng ký - đăng nhập, dữ liệu đã mã hóa và lịch sử hội thoại của người dùng đều sẽ được lưu trữ vào cơ sở dữ liệu phía backend.

1.3.3. Lưu lịch sử hội thoại

Khi người dùng cũ đăng nhập vào tài khoản từng hội thoại, toàn bộ dữ liệu về lịch sử hội thoại sẽ được tải và hiển thị lên UI.

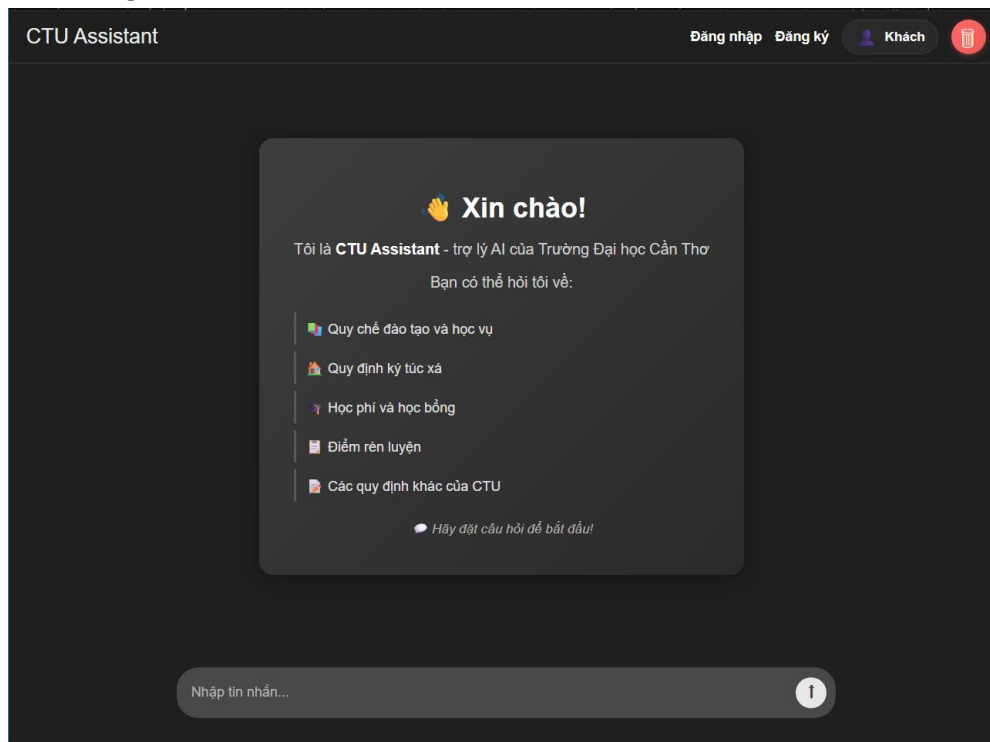
Lịch sử hội thoại cũng đóng vai trò là một phần của ngữ cảnh: Khi có lịch sử hội thoại thì LLM sẽ trả lời tốt hơn, đúng ngữ cảnh hơn, trích xuất và chọn lọc thông tin chính xác hơn.

1.4. Công nghệ sử dụng

- Backend: Python, FastAPI
- AI: Llama3.1:8b, LangChain
- Tìm kiếm: FAISS + BM25
- Database: MySQL
- Frontend: HTML, CSS, JS

1.5. Kết quả đạt được

- Chatbot được xây dựng trên yêu cầu tối thiểu nhất và hoạt động khá tốt: trả lời được chính xác 76% các câu hỏi, trả lời thiếu một phần thông tin 8% các câu hỏi với thời gian phản hồi trung bình là 5.13s và thời gian hoàn tất câu trả lời là 33.41s.
- Hệ thống chạy được trên local ổn định.
- Một giao diện web hoàn chỉnh.



Hình 1-1 Giao diện bắt đầu của trang web

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Trí tuệ nhân tạo và chatbot

2.1.1. Chatbot là gì?

Định nghĩa: Chatbot là chương trình máy tính, là trợ lý ảo dựa trên các thuật toán học sâu của lĩnh vực trí tuệ nhân tạo (AI) được thiết kế để tương tác với người dùng bằng cách sinh ngôn ngữ tự nhiên – mô phỏng quá trình trò chuyện hoặc tương tác giữa người với người.

Phân loại: Chatbot dựa trên nguyên tắc (Rule-based): hoạt động dựa trên các nguyên tắc và các luật được lập trình sẵn và trả lời các câu hỏi đơn giản và có giới hạn. Chúng không thể tự học và phát triển theo thời gian. (VD: các hệ thống tư vấn FAQ đơn giản,...)

Chatbot dựa trên trí tuệ nhân tạo (AI-based): được phát triển dựa trên mô hình học sâu (Deep Learning) và xử lý ngôn ngữ tự nhiên (NLP) để sinh phản hồi các thông tin phức tạp, hiểu ngữ cảnh. Có khả năng tự học để phát triển theo thời gian.(VD: ChatGPT, Gemini, Claude,...đều thuộc nhóm này)

Chatbot truy xuất tăng cường (RAG chatbot): một nhánh đặc biệt của AI-based, kết hợp giữa khả năng tìm kiếm tri thức và sinh phản hồi. Khi nhận câu hỏi, chatbot sẽ tiến hành tìm kiếm thông tin, lấy thông tin này làm ngữ cảnh (context) cho LLM và LLM sẽ sinh phản hồi dựa trên thông tin được cung cấp. Đây là phân loại được sử dụng cho đề tài.

2.1.2. Ứng dụng của chatbot

Chatbot hiện nay được ứng dụng khá rộng và đa lĩnh vực nhờ khả năng tương tác động, xử lý ngôn ngữ tự nhiên, phản hồi nhanh và hiểu ngữ cảnh. Có thể kể đến một số lĩnh vực như:

- Chăm sóc khách hàng (Customer Service)
- Giáo dục (Education)
- Thương mại điện tử (E-Commerce)
- Y tế (Healthcare)
- Hành chính công (Public Service)
- Ứng dụng nội bộ (Enterprise/Organization)

2.2. Mô hình ngôn ngữ lớn (LLM)

Mô hình ngôn ngữ lớn (LLM – Large Language Model) là mô hình trí tuệ nhân tạo được học từ lượng lớn các văn bản đa ngôn ngữ dựa trên các giải thuật học sâu (Deep Learning) – đặc biệt là kiến trúc mạng nơ-ron nhân tạo (NN – Neural Network)

và thường là theo mô hình Transformer để có thể nắm được cấu trúc và ý nghĩa của ngôn ngữ. Từ đó, LLM có thể dự đoán được từ tiếp theo trong câu và sinh ra câu trả lời mạch lạc, hợp ngữ cảnh như con người.

Các LLM được sử dụng phổ biến ở hiện tại: GPT (OpenAI), Claude (Anthropic), Gemini (Google), LLaMA (Meta),...

Đề tài sử dụng mô hình LLaMA3.1:8b (phiên bản mã nguồn mở của Meta) vì nó hoàn toàn miễn phí và chạy được local, offline trên máy tính cá nhân. Ngoài ra, LLaMA được tích hợp vào thư viện LangChain giúp việc thực hiện các công đoạn trong quá trình phát triển trở nên dễ dàng và đơn giản hơn. Trong đề tài, LLM này đóng vai trò sinh ngôn ngữ tự nhiên để phản hồi người dùng sao cho hợp ngữ cảnh.

2.3. RAG (Retrieval-Augmented Generation)

Vấn đề: LLM được huấn luyện từ hàng tỉ dữ liệu chung chung nhưng lại không có dữ liệu riêng của quy chế từ Đại học Cần Thơ.

Giải pháp:

- Tìm kiếm (Retrieval) tài liệu liên quan
- Đưa những đoạn tài liệu đó vào LLM với vai trò như ngữ cảnh (Augment)
- Sinh câu trả lời dựa trên ngữ cảnh (Generation)

Vấn đề và giải pháp ở trên cũng là cách ngắn gọn nhất để hiểu về kĩ thuật RAG. Ta có thể hiểu quá trình ấy qua các bước như sau:

- **Create Vector database:** chuyển đổi toàn bộ dữ liệu tri thức thành các vector đa chiều và lưu trữ chúng vào vector.
- **User input:** User input một câu truy vấn (query) bằng ngôn ngữ tự nhiên nhằm tìm kiếm câu trả lời.
- **Information Retrieval:** Cơ chế retrieval (đối với dự án này) - tiến hành chuyển đổi query người dùng thành vector, sau đó tính khoảng cách của vector này với các Vector database có sẵn và lấy ra số lượng nhất định các vector gần nhất với nó (tức gần nhau nhất về mặt ngữ nghĩa) cuối cùng gửi đến LLM để làm tăng context cho quá trình sinh câu trả lời.
- **Combining data:** sau quá trình retrieval, kết quả của nó sẽ được nối thêm với các thông tin khác như lịch sử hội thoại, system prompt... và query ban đầu của người dùng tạo thành một prompt template.
- **Generate text:** Prompt template gửi đến LLM để sinh phản hồi cuối cùng.

2.4. Tìm kiếm thông tin

2.4.1. Vector Embedding

Định nghĩa: Là phương pháp chuyển đổi các thông tin, các dữ liệu sang vector số trong không gian đa chiều, nhằm biểu diễn ý nghĩa hay mối quan hệ về mặt ngữ nghĩa của dữ liệu trong không gian. Các dữ liệu có ngữ nghĩa giống nhau sẽ ở gần nhau hơn trong không gian vector này.

Lợi ích: Việc vector embedding dữ liệu sẽ giúp nâng cao hiệu suất của mô hình và hiệu quả truy suất, tăng độ chính xác khi tìm kiếm thông tin (semantic search), có thể hiểu được ngữ nghĩa thay vì chỉ “keyword”, ngoài ra còn giúp giảm nhiễu trong tập dữ liệu lớn. Hơn thế nữa, việc embedding có thể tái sử dụng cho nhiều dự án khác mà không cần làm lại quá trình tiền xử lý thông tin.

2.4.2. FAISS (Tìm kiếm theo ngữ nghĩa)

Định nghĩa: Là một thư viện được phát triển bởi Facebook AI Research. Sử dụng tìm kiếm theo độ tương đồng (Similarity search) và gom cụm (Clustering) các vector. Nói tóm gọn, FAISS hỗ trợ chuyên về về phân lưu trữ và truy suất khoảng cách các vector embedding.

Khả năng và vấn đề của FAISS: Hiểu nghĩa, tìm vector dựa theo độ tương đồng ngữ nghĩa trong không gian vector đa chiều (dự án sử dụng model sinh 768 chiều).

2.4.3. BM25 (Tìm kiếm theo từ khóa)

- **Định nghĩa:** BM25 (Best Matching 25) là thuật toán xếp hạng truy xuất thông tin (Information Retrieval Ranking Function) thuộc họ mô hình xác suất. Nó đánh giá mức độ liên quan giữa truy vấn và tài liệu dựa trên tần suất xuất hiện từ khóa (term frequency) và độ hiếm của từ.
- **Khả năng và vấn đề của BM25:** Tìm nhanh – cho kết quả ổn định, dựa theo độ lặp “keyword” nhưng không hiểu ngữ nghĩa, phụ thuộc mạnh vào tokenization và stopwords.

2.4.4. Hybrid Search (tìm kiếm lai / tìm kiếm kết hợp)

Kết hợp giữa FAISS và BM25, đưa ra kết quả chính xác và đúng ngữ nghĩa.

2.5. Các công nghệ sử dụng

- Vector Embedding → Sentence Transformer (Hugging-Face) model `bkai-foundation-models/vietnamese-bi-encoder` chuyên về tiếng Việt
- LangChain → Tiền xử lý dữ liệu, Template, logic RAG
- FAISS + BM25 → Engine tìm kiếm

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Yêu cầu hệ thống (System Requirements)

3.1.1. Yêu cầu chức năng (Functional Requirements)

Yêu cầu chức năng là những gì hệ thống phải có và phải làm được:

- Người dùng có thể đăng ký, đăng nhập, sử dụng chế độ khách và đăng xuất.
- Người dùng có thể gửi câu hỏi đến chatbot.
- Người dùng có thể xem và xóa lịch sử trò chuyện.
- Hệ thống phải tiếp nhận truy vấn và tiến hành tìm kiếm thông tin liên quan, trích xuất thông tin. Sau đó gửi LLM sinh câu trả lời và phản hồi người dùng.

3.1.2. Yêu cầu phi chức năng (Non-Functional Requirements)

Yêu cầu phi chức năng là các khả năng đặc trưng để mô tả chất lượng của hệ thống:

- **Hiệu năng:** Thời gian phản hồi trung bình phải < 5s với các truy vấn ngắn
- **Khả năng mở rộng:** Có thể dễ dàng thêm các tài liệu mới
- **Bảo mật:** Mật khẩu của người dùng phải được mã hóa.
- **Triển khai:** Chạy được trên local và không cần internet, chỉ yêu cầu VRAM và GPU hoặc CPU.
- **Giao diện:** thân thiện người dùng, reponsive hợp lí.

3.1.3. Yêu cầu phần cứng và phần mềm

a. Yêu cầu phần cứng (Hardware Requirements)

Bảng 3-1 Yêu cầu phần cứng

Loại	Mức thấp (chạy ổn định)	Mức cao (hiệu suất tốt nhất)	Ghi chú
CPU	CPU	Intel core i5 đời 10 trở lên hoặc Ryzen 5 3600 trở lên	Càng nhiều nhân thì khả năng RAG và response API càng nhanh
RAM	8GB	64GB	RAM ảnh hưởng trực tiếp đến tốc độ embedding và tìm kiếm các vector embedding
GPU	Không bắt buộc (CPU only)	NVIDIA RTX 4080 (16GB VRAM) trở lên	Giúp tăng tốc độ xử lý LLM với các model lớn và nhiều tham số (VD: 70B tham số)

	Hoặc chỉ cần NVIDIA RTX 3040 4GB		
Ổ cứng	Trống nhiều hơn 15GB	Trống nhiều hơn 50GB	Lưu trữ LLM cục bộ, Vector embedding và Database

b. Yêu cầu phần mềm (Software Requirements)

Bảng 3-2 Yêu cầu phần mềm

Thành phần	Phiên bản	Ghi chú
Hệ điều hành	Ubuntu / Window 10+ / macOS 14+	Ubuntu ổn định cho các model lớn
Thư viện	Các phiên bản được cài đặt tự động trong file requirements.txt hoặc mới nhất	Phiên bản được tự động trong file sẽ ổn định
Trình duyệt	Chrome (khuyến dùng) / Edge / Firefox bản mới nhất	Bản mới nhất sẽ tương thích tốt và hỗ trợ đầy đủ nhất

c. Môi trường phát triển hệ thống

Khuyến nghị nên cài đặt môi trường ảo (venv hoặc conda) để cài đặt các tài nguyên và thư viện cần thiết, tránh cài đặt thẳng vào ổ C gây hao dung lượng và gây khó trong quá trình xóa bỏ.

Trình soạn thảo (có hỗ trợ extension) hoặc IDE (Integrated Development Environment) khuyến dùng là Visual Studio Code và Pycharm.

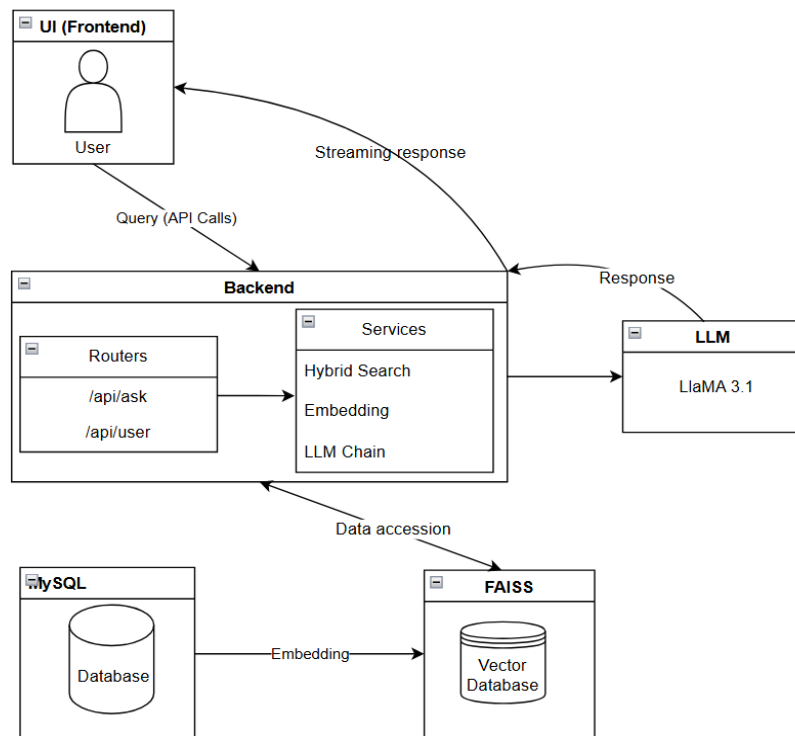
3.2. Kiến trúc hệ thống

3.2.1. Tổng quan

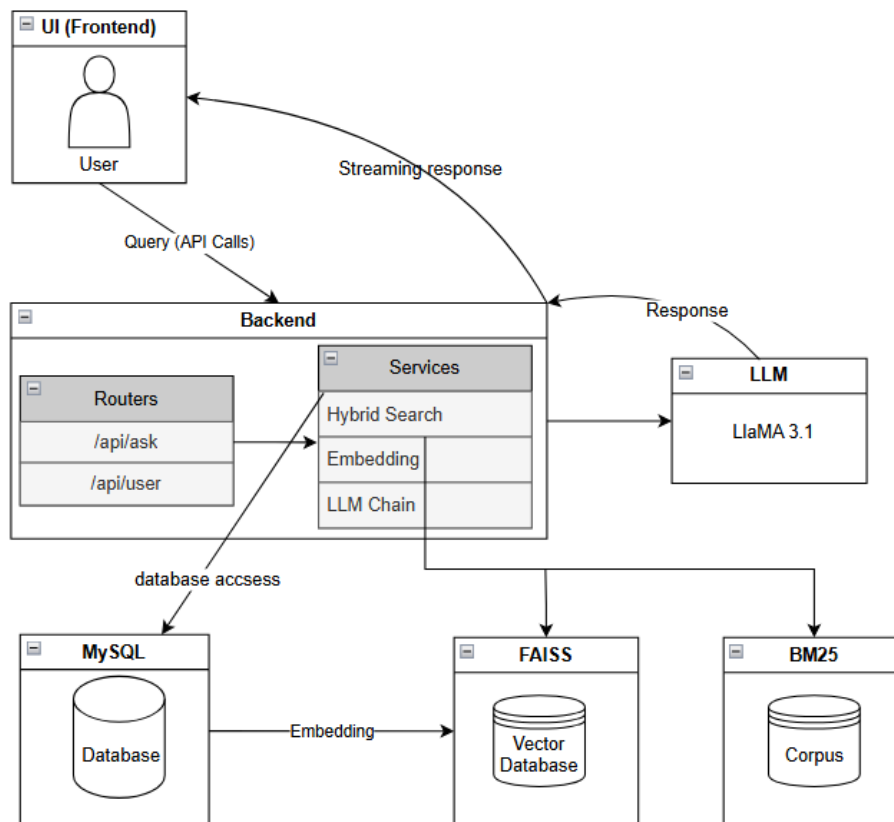
Hệ thống chatbot hỗ trợ tra cứu quy định của Đại học Cần Thơ được xây dựng dựa trên kiến trúc client-server, gồm 3 lớp chính:

- **Frontend (Giao diện người dùng):** Hiển thị hộp thoại chat (chatbox), nơi nhận câu hỏi từ người dùng và hiển thị phản hồi từ chatbot
- **Backend (Phía server - FastAPI):** Xử lý các yêu cầu, điều phối các module tìm kiếm, phản hồi người dùng và quản trị dữ liệu. Ngoài ra, backend còn đóng vai trò là một “client” khác để gửi request đến LLM chạy trên local (nhận vai trò như 1 server).
- **Cơ sở dữ liệu (MySQL):** Lưu trữ thông tin người dùng, chuỗi hội thoại và các tập dữ liệu dùng để tìm kiếm.

3.2.2. Sơ đồ kiến trúc hệ thống



Hình 3-1 Kiến trúc tổng quan của hệ thống



Hình 3-2 Sơ đồ kiến trúc hệ thống

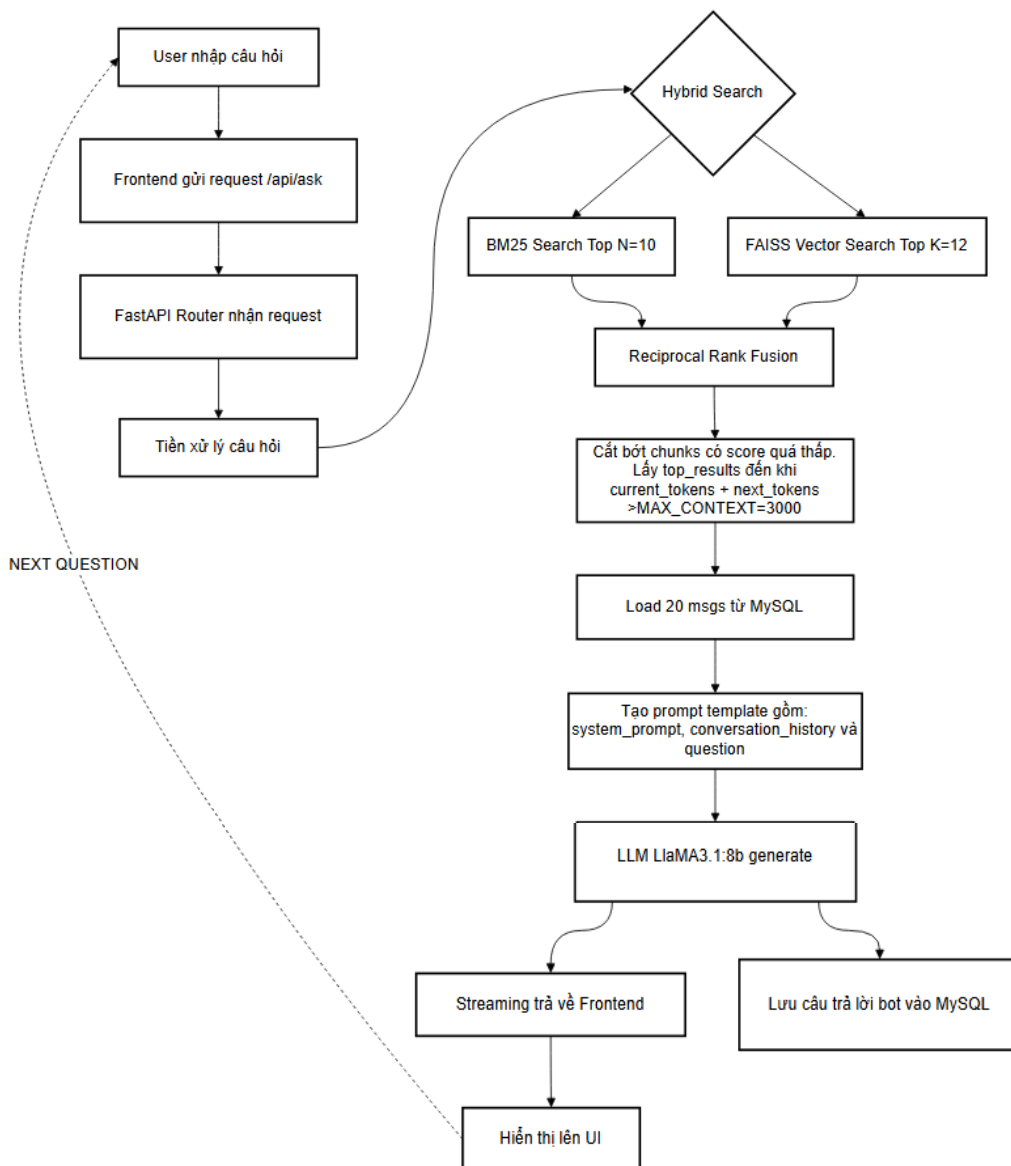
Hệ thống được thiết kế theo mô hình gồm 3 lớp chính:

- Lớp giao diện (UI Frontend): người dùng tương tác qua trình duyệt web.
- Lớp xử lý nghiệp vụ (Backend): FastAPI với các services xử lý RAG
- Lớp dữ liệu: MySQL (lưu dữ liệu hệ thống, messages, users) và FAISS (vector embeddings)

Khi người dùng gửi câu hỏi, hệ thống:

- Nhận yêu cầu từ router /api/ask
- Services thực hiện Hybrid Search (FAISS + BM25) và truy suất nội dung qua meta_data
- LLM Chain tổng hợp context và gọi LlaMA3.1
- Trả kết quả cho user và lưu vào Database

3.2.3. Luồng hoạt động của hệ thống



Hình 3-3 Sơ đồ luồng xử lý câu hỏi

Hình 3-2 mô tả chi tiết luồng xử lý một câu hỏi từ người dùng trong hệ thống chatbot.

Quá trình được chia thành các bước sau:

Bước 1: Nhận yêu cầu từ người dùng:

- User nhập câu hỏi vào giao diện chat
- Frontend gửi HTTP POST request đến endpoint /api/ask với payload:

```
{  "question": "Sinh viên ktx được dùng bao nhiêu điện?",  "user_id": 123,  "conversation_id": 456}
```

Bước 2: Tiền xử lý câu hỏi (Query Preprocessing):

- Chuẩn hóa văn bản: lowercase, remove special characters
- Mở rộng từ đồng nghĩa: "ktx" → "ký túc xá"

Bước 3: Tìm kiếm tài liệu (Hybrid Search):

- Hệ thống thực hiện song song 2 phương pháp tìm kiếm:
- **3a. BM25 Search (Keyword-based):**
 - Tìm kiếm dựa trên từ khóa trong câu hỏi
 - Trả về top 10 documents có điểm BM25 cao nhất
- **3b. FAISS Vector Search (Semantic-based):**
 - Chuyển câu hỏi thành vector embeddings 768 chiều bằng model (bkaifoundation-models/vietnamese-bi-encoder)
 - Tìm kiếm trong 2 index (đã được vector embeddings từ trước) song song:
 - + index.faiss: Tìm trong vectors có dấu
 - + index_none_accent.faiss: Tìm trong vectors không dấu
 - Trả về top 10 documents gần nhất (L2 distance)
- Tiến hành tính điểm và lấy các chunks có top cao nhất:
- **3c. Reciprocal Rank Fusion (RRF)**
 - Kết hợp kết quả từ BM25 và FAISS
 - Công thức: $score(d) = \sum (1/(k + rank_i(d)) * \text{trọng số})$ với $k=50$
 - Trọng số: BM25 (0.6), FAISS (0.4)
 - Lọc chunks có $score < 0.005$ (loại bỏ kết quả không liên quan)
 - Chọn chunks theo thứ tự điểm cao nhất cho đến khi:
 - + Tổng tokens đạt MAX_CONTEXT=3000 tokens hoặc

- + Hết candidates
- Output: Động (thường 5-8 chunks, tùy độ dài mỗi chunk)

Ví dụ:

- Nếu mỗi chunk ~500 tokens → lấy được ~6 chunks
- Nếu chunk đầu tiên đã 2800 tokens và chunks thứ 2 lớn hơn 200 tokens → chỉ lấy được 1 chunks
- Lý do dùng MAX_CONTEXT thay vì top-K cố định:
 - + Tránh vượt quá giới hạn context window của LLM (8192 tokens)
 - + Giữ lại budget cho conversation history (~20 messages)
 - + Tối ưu chất lượng: Ưu tiên chunks ngắn gọn, đúng trọng tâm

Bước 4: Xây dựng context cho LLM

- Load 20 tin nhắn gần nhất từ database (10 cặp hỏi-đáp)
- Format conversation history theo string:

```
"User: [câu hỏi 1]
Bot: [câu trả lời 1]
User: [câu hỏi 2]
Bot: [câu trả lời 2]
..."
```
- Kết hợp với context trả về từ Hybrid Search
- Kết hợp system_prompt: Hình dưới đây là nguyên tắc được đặc ra cho LLM

```
_system_prompt = """
Bạn là CTU Assistant - trợ lý AI của Trường Đại học Cần Thơ (CTU).

Mục tiêu:
- Giải đáp chính xác, ngắn gọn, dựa trên quy định, chính sách, hoặc nội quy của CTU.
- Duy trì mạch hội thoại hợp lý giữa các câu hỏi liên tiếp.

Nguyên tắc:
1. Ưu tiên thông tin từ "Lịch sử hội thoại" nếu câu hỏi có liên quan đến các lượt trước.
2. Ưu tiên dữ liệu từ phần "Tài liệu tham khảo" để trích dẫn hoặc xác nhận.
3. Nếu hai nguồn mâu thuẫn, tin "Tài liệu tham khảo" hơn.
4. Nếu không có thông tin trong tài liệu, hãy nói rõ: "Tôi không tìm thấy thông tin liên quan trong quy định của CTU."
5. Trả lời rõ ràng, tối đa 3 câu, không lặp lại nội dung câu hỏi.
6. Nếu câu hỏi yêu cầu tính toán (ví dụ: điểm, học bổng...), chỉ suy luận dựa trên thông tin người dùng đã cung cấp trong hội thoại.
7. Nếu câu hỏi không đủ dữ kiện, hãy hỏi lại để làm rõ thay vì trả lời chung chung.

Phong cách trả lời:
- Ngắn gọn, súc tích, đúng trọng tâm.
- Nếu hội thoại đã có ngữ cảnh, chỉ trả lời phần mới hoặc kết luận logic.
"""
```

Hình 3-4 System_prompt cho LLM

- Build prompt template: Hình bên dưới là prompt template gửi cho LLM

```
template = [
    ("system", _system_prompt),
    ("user",
     "Lịch sử hội thoại gần đây:\n{conversation_history}\n\n"
     "Tài liệu tham khảo (chỉ chọn phần liên quan, có thể chứa thông tin dư):\n{content}\n\n"
     "Câu hỏi mới: {question}\n\n"
     "→ Hãy trả lời ngắn gọn (1-3 câu), logic theo ngữ cảnh hội thoại, chỉ dựa vào thông tin có thật.")
]
```

Hình Error! Use the Home tab to apply 0 to the text that you want to appear here.-6 Prompt template cho LLM

Bước 5: Sinh câu trả lời (LLM Generation):

- Gửi prompt đến Llama 3.1(8B) thông qua thư viện langchain_ollama.llms
- LLM đọc context và sinh câu trả lời
- Sử dụng streaming mode: Trả lời từng token (từng từ)

Bước 6: Streaming response

- Backend gửi response về frontend dạng Server-Sent Events (SSE)
- Frontend nhận từng chunk và hiển thị real-time (như ChatGPT)

Bước 7: Lưu trữ vào database

- Sau khi LLM hoàn thành, lưu conversation vào Database theo các thuộc tính:

```
user_id: ID của user
conversation_id: ID của conversation (None để tạo mới)
message: Tin nhắn từ user
reply: Câu trả lời từ bot
```

- Update conversation timestamp

Bước 8: Hiển thị kết quả

- Frontend render câu trả lời vào chat bubble
- User có thể tiếp tục hỏi (quay lại Bước 1)

Thời gian xử lý trung bình:

- Query Preprocessing: ~0.05s
- Hybrid Search: ~0.3s
- LLM Generation: ~12-77s (tùy độ dài câu trả lời)
- Tổng: ~12.5-77.5s

Ưu điểm của luồng xử lý:

- Hybrid Search tăng độ chính xác (keyword + semantic)
- Dual FAISS index xử lý tốt input thiếu dấu
- Conversation history giúp hiểu ngữ cảnh
- RAG giảm hallucination (LLM trả lời dựa trên tài liệu)
- Streaming response cải thiện UX (user không phải chờ lâu)

3.2.4. Ưu điểm kiến trúc

Kiến trúc hệ thống được thiết kế theo mô hình 3 lớp (Frontend – Backend – Database) kết hợp với cơ chế RAG và Hybrid Search mang lại các ưu điểm:

1. Tách biệt rõ ràng các phần

Frontend, Backend và Database hoạt động độc lập nhau giúp dễ mở rộng, dễ bảo trì, giảm rủi ro khi nâng cấp từng phần.

2. Hiệu năng cao

Hybrid Search (FAISS + BM25) tăng độ chính xác nhờ kết hợp tìm kiếm ngữ nghĩa và từ khóa, xử lý nhanh trên dữ liệu lớn, giảm độ trễ khi phản hồi.

3. Giảm ảo giác thông tin và sự sai lệch của LLM (hallucination)

LLM chỉ sinh câu trả lời dựa trên context được truy xuất từ tài liệu thật nên thông tin là chính xác và phù hợp với quy định của trường.

4. Hỗ trợ tốt tiếng Việt và dữ liệu đa dạng

Hai FAISS index (có dấu và không dấu) cùng với bảng mở rộng ký tự đồng nghĩa (synonyms) giúp xử lý tốt các câu truy vấn không chuẩn, viết tắt hoặc thiếu dấu.

5. Khả năng mở rộng linh hoạt

Có thể thêm tài liệu mới, mở rộng vector database, thay model LLM hoặc nâng cấp hạ tầng mà không ảnh hưởng hệ thống hiện tại.

6. Tối ưu trải nghiệm người dùng

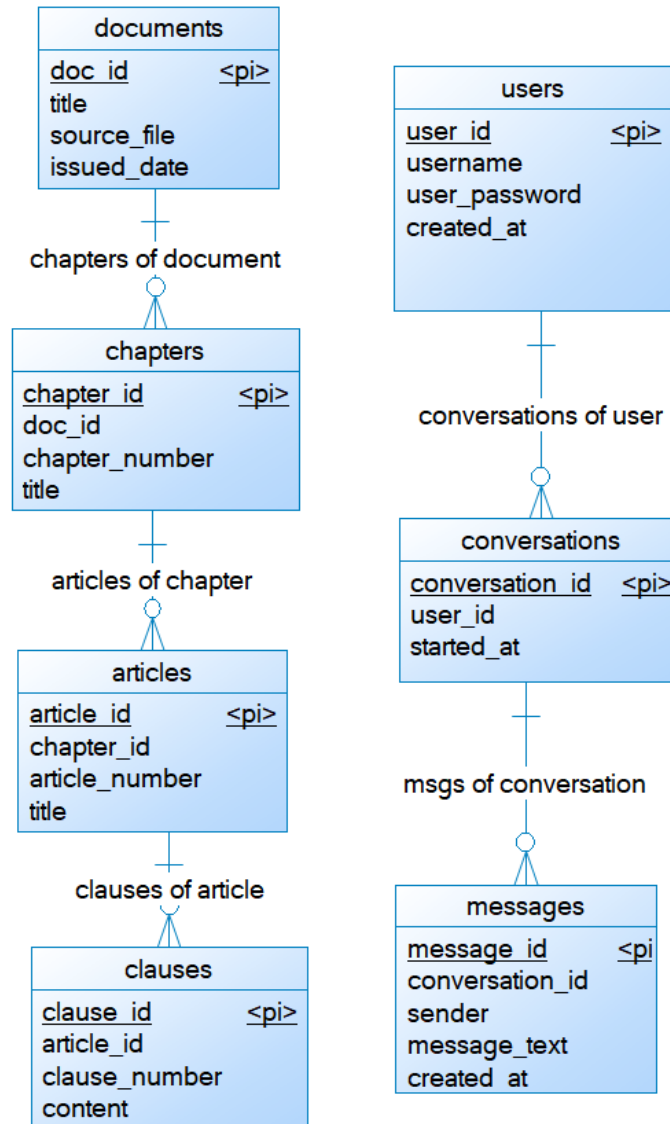
Kiến trúc hỗ trợ streaming response giúp người dùng thấy câu trả lời theo thời gian thực, giảm cảm giác chờ đợi.

7. An toàn dữ liệu và dễ quản lý

MySQL lưu toàn bộ user, conversation và data gốc, hỗ trợ xác thực, phân quyền và sao lưu ổn định.

3.3. Thiết kế cơ sở dữ liệu (Database)

3.3.1. Sơ đồ thực thể - quan hệ (ERD)



Hình 3-6 Sơ đồ thực thể - quan hệ của hệ thống

Hình 3-5 Mô tả các thực thể và quan hệ được sử dụng trong hệ thống, gồm 2 nhóm chính độc lập nhau:

- Nhóm dữ liệu các quy định sử dụng cho tìm kiếm:

Documents (tài liệu) [1-N] → Chapter (chương) [1-N] → articles (điều) [1-N] → clauses (khoản / nội dung) xây dựng theo cấu trúc phân cấp cha-con, mỗi thành phần con chỉ thuộc đúng 1 cha tức là một tài liệu gồm một hoặc nhiều chương, mỗi chương gồm một hoặc nhiều điều và mỗi điều gồm một hoặc nhiều khoản/nội dung. Nhóm này được sử dụng cho HybridSearch/RAG.

- Nhóm dữ liệu người dùng và hội thoại:

Users (người dùng) [1-N] → Conversations (chuỗi hội thoại) [1-N] → Messages (tin nhắn) xây dựng theo cấu trúc phân cấp cho phép một người dùng có thể có nhiều chuỗi hội thoại khác nhau với chatbot và mỗi tin nhắn có “sender” thuộc về “user” hoặc “bot”. Nhóm này lưu dữ liệu người dùng và các chuỗi hội thoại phục vụ cho đăng nhập và tạo ngữ cảnh cho mô hình.

3.3.2. Mô tả các bảng chính

Các mục dưới đây sẽ mô tả chi tiết hơn về cách thiết kế cơ sở dữ liệu của từng bảng được sử dụng trong dự án như: khóa chính, khóa ngoại, kiểu dữ liệu...

a. Bảng Documents

Bảng 3-3 Bảng mô tả chi tiết table documents

STT	Thuộc tính	Kiểu dữ liệu	Khóa chính	Khóa ngoại	Mô tả
1	doc_id	Int	x		ID tài liệu (được tạo tự động)
2	title	Text			Tên tài liệu
3	source_file	Varchar(500)			Đường dẫn đến file gốc
4	issued_date	Date			Ngày ban hành

b. Bảng Chapters

Bảng 3-4 Bảng mô tả chi tiết table chapters

STT	Thuộc tính	Kiểu dữ liệu	Khóa chính	Khóa ngoại	Mô tả
1	chap_id	Int	x		ID chương (được tạo tự động)
2	doc_id	Int		x	ID tài liệu liên kết với bảng documents
3	chapter_number	Int			Số thứ tự chương
4	title	Text			Tiêu đề chương

c. Bảng Articles

Bảng 3-5 Bảng mô tả chi tiết table articles

STT	Thuộc tính	Kiểu dữ liệu	Khóa chính	Khóa ngoại	Mô tả
1	article_id	Int	x		ID điều (được tạo tự động)

2	chapter_id	Int		x	ID chương liên kết với bảng chapters
3	article_number	Int			Số thứ tự điều
4	Title	Text			Tiêu đề điều

d. Bảng Clauses

Bảng 3-6 Bảng mô tả chi tiết table clauses

STT	Thuộc tính	Kiểu dữ liệu	Khóa chính	Khóa ngoại	Mô tả
1	clause_id	Int	x		ID khoản/nội dung (được tạo tự động)
2	article_id	Int		x	ID điều liên kết với bảng articles
3	clause_number	Int			Số thứ tự khoản/nội dung
4	Content	Long Text			Nội dung

e. Bảng Users

Bảng 3-7 Bảng mô tả chi tiết table users

STT	Thuộc tính	Kiểu dữ liệu	Khóa chính	Khóa ngoại	Mô tả
1	user_id	Int	x		ID người dùng(được tạo tự động)
2	username	Varchar(100)			Tên đăng nhập (unique)
3	user_password	Varchar(100)			Mật khẩu đã hash (bcrypt)
4	created_at	Timestamp			Ngày tạo tài khoản

f. Bảng Conversations

Bảng 3-8 Bảng mô tả chi tiết table conversations

STT	Thuộc tính	Kiểu dữ liệu	Khóa chính	Khóa ngoại	Mô tả
1	conversation_id	Int	x		ID hội thoại (được tạo tự động)
2	user_id	Int		x	ID người dùng liên kết với bảng users
3	started_at	Timestamp			Thời gian bắt đầu

g. Bảng Messages

Bảng 3-9 Bảng mô tả chi tiết table messages

STT	Thuộc tính	Kiểu dữ liệu	Khóa chính	Khóa ngoại	Mô tả
1	message_id	Int	x		ID tin nhắn(được tạo tự động)
2	conversation_id	Int		x	ID chuỗi hội thoại liên kết với bảng conversations
3	sender	Enum('user', 'bot')			Người gửi là người dùng hay bot
4	message_text	Text			Nội dung tin nhắn
5	created_at	Timestamp			Thời gian gửi

3.3.3. Cơ sở dữ liệu vector (FAISS Index)

Trước khi lưu và sử dụng, cần chuẩn hóa cơ sở dữ liệu sao cho hợp lý nhất:

- Làm sạch từ: xóa ký tự đặc biệt.
- Tạo danh sách câu dựa vào “keyword” có từ đồng nghĩa.
- Loại bỏ dấu câu: đối với faiss index non-accent sẽ được nhắc ở phần sau.
- Đối với FAISS index, ta không xóa stopwords bởi vì khi tìm kiếm bằng ngữ nghĩa, đôi lúc stopwords lại có ích cho tìm kiếm ngữ nghĩa.

Ví dụ:

User query: “Tôi học được 9.0 thì thang điểm 4 của tôi được bao nhiêu và có được loại giỏi không á?????”

Sau khi chuẩn hóa: [“tôi học được 9.0 thì thang điểm 4 của tôi được bao nhiêu và có được loại giỏi không”, “tôi học được 9.0 thì điểm 4 của tôi được sao và có được học lực giỏi không”, “tôi học được 9.0 thì điểm chữ của tôi được sao và có được phân loại giỏi không” ...]

Cơ sở dữ liệu vector lưu các chunk embeddings (hiện tại có 183 chunks) tuân theo các điều kiện sau:

- Mỗi chunk là một “clause”
- Mỗi clause không được dài hơn 750 tokens, chunk_overlap = 120
- Nếu dài hơn 750 tokens, tiến hành cắt chunk theo độ ưu tiên: “\nĐiều”, “\nNhóm”, “\n1.”, “\na)”, “\nn”, “\n”

Dưới đây là một đoạn dữ liệu mô tả trực quan thực tế một tài liệu được tách chunks như thế nào:

```
[Doc 9] QUY ĐỊNH KỶ TÚC XÁ ĐẠI HỌC CẦN THƠ
├ [Chap 16] QUY ĐỊNH KỶ TÚC XÁ ĐẠI HỌC CẦN THƠ
│   ├── [Arti 124] Điều 1. Đối tượng áp dụng (1 clauses, 94 tokens)
│   │   └ [1] Nội quy này áp dụng cho tất cả sinh viên các hệ đào tạo tại...
│   ├── [Arti 125] Điều 2. Một số quy định chung (1 clauses, 361 tokens)
│   │   └ [1] Sinh viên muốn vào ở nội trú tại ký túc xá phải thực hiện đầ...
│   ├── [Arti 126] Điều 3. Quy định về sinh hoạt (1 clauses, 252 tokens)
│   │   └ [1] Sinh viên khi ra vào khu ký túc xá phải mang thẻ sinh viên v...
│   ├── [Arti 127] Điều 4. Quy định về sử dụng phòng ở (1 clauses, 309 tokens)
│   │   └ [1] Không di chuyển giường, tủ và vật dụng trong phòng. Không tự...
│   ├── [Arti 128] Điều 5. Công tác vệ sinh môi trường và c (1 clauses, 75 tokens)
│   │   └ [1] Sinh viên phải vệ sinh, sắp xếp ngăn nắp phòng ở, khu vực xu...
│   ├── [Arti 129] Điều 6. Giữ gìn an ninh trật tự (1 clauses, 184 tokens)
│   │   └ [1] Sinh viên phải có trách nhiệm giữ gìn an ninh trật tự, an to...
│   ├── [Arti 130] Điều 7. Quy định về sử dụng nhà xe ký túc (1 clauses, 174 tokens)
│   │   └ [1] Nhà xe phục vụ sinh viên nội trú. Sinh viên gửi xe đạp miễn...
│   ├── [Arti 131] Điều 8. Quy định về sử dụng điện, nước v (1 clauses, 113 tokens)
│   │   └ [1] Mỗi phòng có đồng hồ đo điện, nước riêng. Sinh viên trả phí...
│   ├── [Arti 132] Điều 9. Khen thưởng và kỷ luật (1 clauses, 105 tokens)
│   │   └ [1] Tập thể hoặc cá nhân sinh viên thực hiện tốt nội quy, có thà...
│   └ [Arti 133] Điều 10. Bảng Nội dung vi phạm và khung (3 clauses, 1639 tokens)
│       ├── [1] Nhóm 1: VI PHẠM NGHIÊM TRỌNG (Buộc ra khỏi KTX ngay lập tức)...
│       ├── [2] Nhóm 4: VI PHẠM VỀ NẤU ĂN 1. Cất giữ dụng cụ nấu ăn trong ph...
│       └ [3] Nhóm 7: VI PHẠM VỀ VỆ SINH MÔI TRƯỜNG 1. Không giữ vệ sinh p...
```

3.3.4. Cơ sở dữ liệu ngữ nghĩa (BM25 Index)

BM25 index thực tế dự án không lưu thành một file .pkl cơ sở dữ liệu phục vụ lâu dài mà dự án tiến hành “xây lại” index từ database(SQL) mỗi lần khởi động lại server. Nhưng cả 2 việc lưu file và không lưu file có một điểm chung là cần làm sao để thiết kế một “corpus” sao cho sạch nhất phục vụ dự án:

Trước khi sử dụng corpus, cần tiến hành:

- Loại bỏ ký tự đặc biệt.
- Phân từ(segment word). (VD: có thể sử dụng thư viện *underthesea*)
- Loại bỏ từ thừa (stopword). (VD: á, í, à, sao...)

Ví dụ:

User query: “Tôi học được 9.0 thì thang điểm 4 của tôi được bao nhiêu và có được loại giỏi không á?????”

Sau khi chuẩn hóa: “học được 9.0 thang_điểm 4 được bao_nhiêu được loại_giỏi”

Sau khi split: [“học”, “được”, “9.0”, “thang_điểm”, “4”, “được”, “bao_nhiêu”, “được”, “loại_giỏi”].

3.4. Thiết kế hệ thống tìm kiếm

3.4.1. Tổng quan Hybrid Search

Hệ thống kết hợp hai kỹ thuật BM25 (lexical) + FAISS (semantic):

- BM25 tìm từ khóa chính xác, hiệu quả với các truy vấn rõ ràng
- FAISS tìm theo ngữ nghĩa bằng vector embeddings 768 chiều.

Hai phương pháp trên được hợp nhất bằng Reciprocal Rank Fusion (RRF) để tối ưu kết quả của cả tìm kiếm bằng từ khóa lẫn ngữ nghĩa.

3.4.2. BM25 Index

Quy trình: Quy trình chuẩn hóa tuân theo các mục đã nói ở phần cơ sở dữ liệu ngữ nghĩa trên.

Tham số: mặc định toàn bộ.

Kích thước dữ liệu: khoảng 183 chunks từ 14 văn bản quy định.

Thuật toán BM25 có công thức như sau:

$$\text{score}(d, q) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) \cdot (k_1 + 1)}{f(t, d) + k_1 \cdot (1 - b + b \cdot \frac{|d|}{\text{avg}_d})}$$

Trong đó:

- $f(t, d)$: tần suất từ
- $|d|$: độ dài tài liệu
- avg_d : độ dài trung bình
- k_1, b : tham số cần điều chỉnh

BM25 tính độ liên quan dựa trên tần suất xuất hiện từ khóa, độ dài tài liệu và mức độ phổ biến của từ. Đề tài sử dụng hai tham số chuẩn là: $k_1 = 1.5$ và $b = 0.75$. Ngoài ra, các tham số sẽ được thư viện python hỗ trợ tính tự động nếu không điều chỉnh.

3.4.3. FAISS Vector Database

Model embedding: bkai-foundation-models/vietnamese-bi-encoder.

Index type: FAISS Flat (L2 distance).

Kích thước vector: 768 (khác nhau tùy thuộc vào model).

Dual index:

- `index.faiss`: vector có dấu.
- `index_none_accent.faiss`: vector không dấu (dùng cho truy vấn sai dấu).

Metadata: `id_mapping.json` chứa thông tin giúp ánh xạ từ `vector_id` → (`doc_id`, `chapter_id`, `article_id`).

Số vectors được lưu trữ trong dự án: khoảng 183 vectors.

FAISS không phải là thuật toán tìm kiếm mà là một thư viện giúp tăng tốc nearest-neighbor search. FAISS lưu các dữ liệu được embeddings dạng vector, dùng chỉ số Flat (dự án dùng L2 distance) để tính khoảng cách và tìm các vector gần nhất bằng cách so sánh khoảng cách giữa chúng.

3.4.4. Chiến lược kết hợp (RRF)

Phương pháp hợp nhất sử dụng Reciprocal Rank Fusion(RRF) có công thức như sau:

$$Score(d) = \sum_{i \in \{BM25, FAISS\}} \frac{W_i}{k + rank_i(d)}$$

Trong đó:

- $\alpha = 0.6$ (BM25)
- $\beta = 0.4$ (FAISS)
- Hằng số RRF: $k = 50$

Quy tắc chọn context:

- Giới hạn context: MAX_CONTEXT = 3000 tokens.
- Loại các chunk có score < 0.005.
- Chọn từ cao $\rightarrow$ thấp đến khi đủ budget.

Ưu điểm khi thực hiện hợp nhất kết quả:

- Tăng độ chính xác cho câu hỏi mơ hồ.
- Giảm rủi ro “miss” những tài liệu quan trọng.
- Tối ưu hóa input khi gửi vào LLM.

3.5. Thiết kế API

3.5.1. Cấu trúc tổng thể

Backend sử dụng **FastAPI**, chia thành 3 lớp chính:

- **routers/**: định nghĩa các endpoint (/api/ask, api/user)
- **services/**: xử lý logic (Hybrid Search, RAG pipeline, preprocessing)
- **db/**: quản lý MySQL connection

Cách chia này giúp code tách biệt, dễ bảo trì và mở rộng.

3.5.2. Danh sách API chính

Bảng 3-10 Bảng danh sách các API chính

STT	Endpoint	Method	Chức năng
1	/api/ask	POST	Gửi câu hỏi để nhận streaming response
2	/api/user/register	POST	Đăng ký

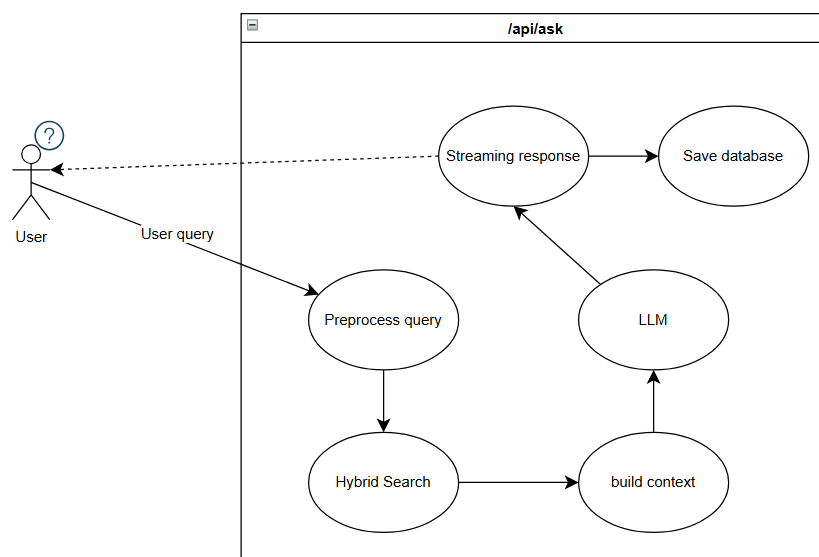
3	/api/ user/login	POST	Đăng nhập
4	/api/ user/conversation/{user_id}	GET	Lấy danh sách hội thoại
5	/api/ user/conversation/{id}	DELETE	Xóa hội thoại

3.5.3. Luồng xử lý API chính (/api/ask)

Input:

```
{
  "question": "Sinh viên KTX được dùng bao nhiêu điện?",
  "user_id": 3,
  "conversation_id": 12
}
```

Sơ đồ luồng xử lý:



Hình 3-7 Luồng xử lý API /api/ask

Dựa vào hình Hình 3-8 ta thấy, API /api/ask có luồng xử lý chính như sau:

Preprocessing → Hybrid Search → Build context → LLM → Streaming → Save DB

Output:

```
data: {"token": "Theo"}
data: {"token": " quy"}
data: {"token": " định"}
...
data: {"done": true}
```

3.5.4. Streaming Response

Hệ thống sử dụng Server-Sent Events (SSE):

- Backend sinh từng token từ LLM
- Frontend nhận token và hiển thị real-time

Ưu điểm: Perceived latency thấp, người dùng không phải chờ đợi đến khi LLM trả lời xong mới thấy câu trả lời.

3.5.5. Ưu điểm thiết kế API

Cách thiết kế API trên làm cho mã nguồn trở nên “sạch” hơn, dễ maintain dễ bảo trì hơn khi mà các chức năng được phân tách ra thành từng file từng thư mục riêng biệt, các endpoint đặt tên dễ hiểu ứng với từng chức năng phục vụ việc mở rộng sự án sau này.

3.6. Thiết kế giao diện

3.6.1. Nguyên tắc thiết kế

Để có một hệ thống đơn giản, dễ sử dụng thì cần phải có một giao diện đáp ứng các tiêu chí sau đây:

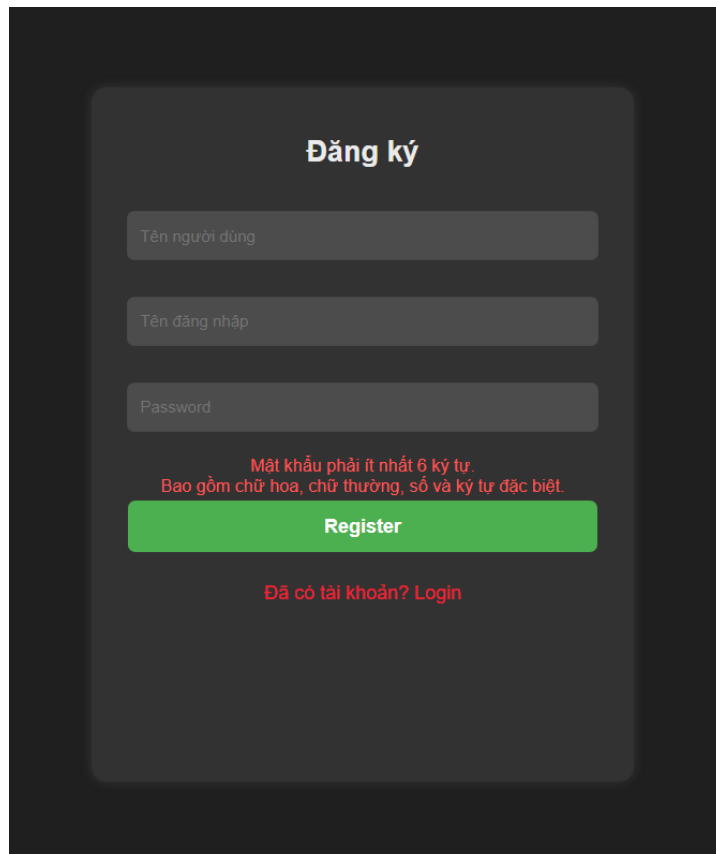
- **Tối giản:** Giao diện chat một cột, tập trung vào chính đoạn chat
- **Responsive:** tương thích với các thiết bị: desktop/laptop/tablet/mobile
- **Real-time:** streaming text, auto-scroll

3.6.2. Giao diện chính

Giao diện hệ thống gồm trang đăng ký, trang đăng nhập, trang chat chính. Thanh navbar của trang chat có các chức năng chính như hiển thị tên người dùng, xóa lịch sử hội thoại và đăng xuất.

a. Trang đăng ký

Trang đăng ký giúp người dùng có thể đăng ký tài khoản gồm các trường cơ bản như tên người dùng, tên đăng nhập và mật khẩu (sẽ được hash bằng bcrypt). Mỗi trường trên đều có form validation trực tiếp mỗi khi có một ký tự được gõ ví dụ như xác thực tồn tại người dùng, xác thực mật khẩu (chữ hoa, thường, ký tự đặc biệt, số...)



Đăng ký

Tên người dùng

Tên đăng nhập

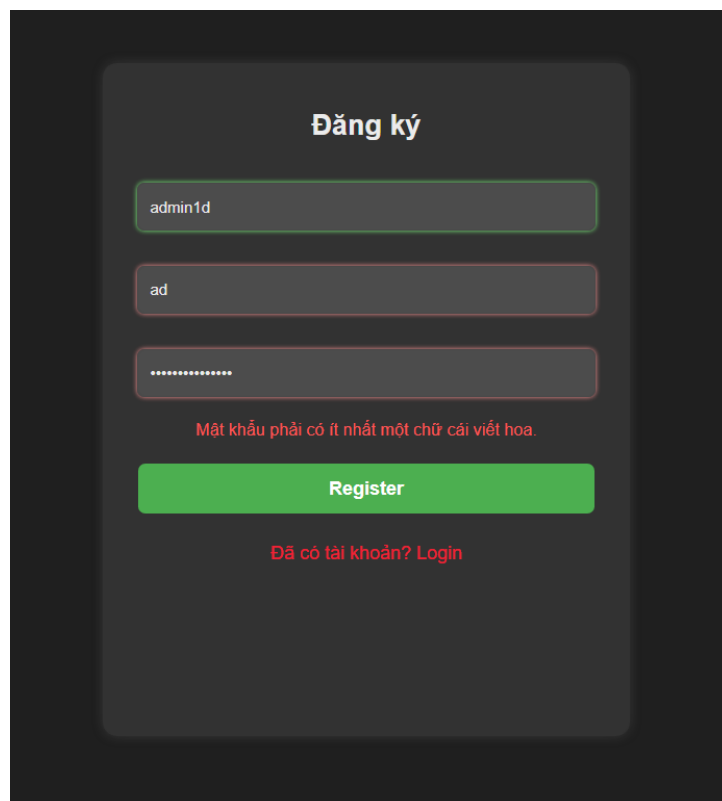
Password

Mật khẩu phải ít nhất 6 ký tự.
Bao gồm chữ hoa, chữ thường, số và ký tự đặc biệt.

Register

Đã có tài khoản? Login

Hình 3-8 Giao diện trang đăng ký



Đăng ký

admin1d

ad

.....

Mật khẩu phải có ít nhất một chữ cái viết hoa.

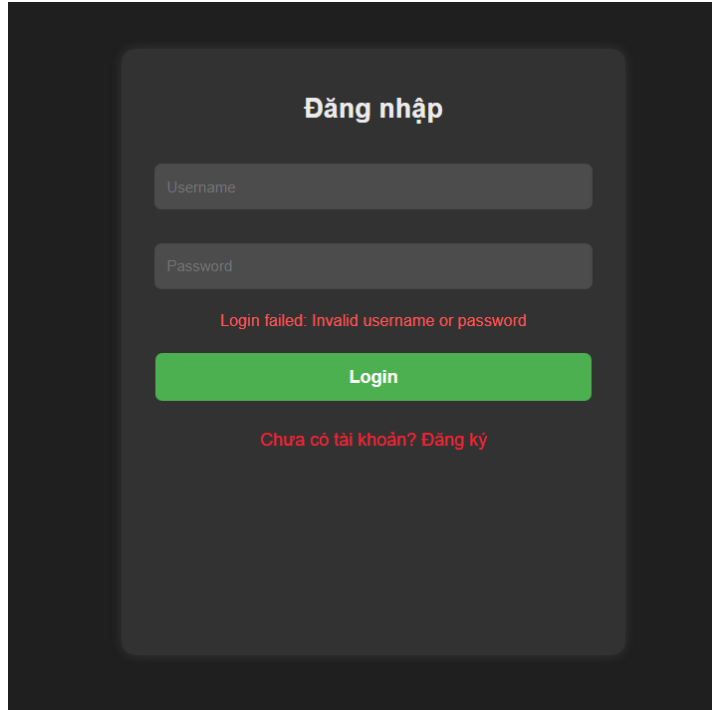
Register

Đã có tài khoản? Login

Hình 3-9 Giao diện trang đăng ký có form validation

b. Trang đăng nhập

Trang đăng nhập giúp người dùng có thể đăng nhập vào hệ thống để chat ở chế độ “người dùng” để có thể lưu lịch sử chat, gồm 2 trường chính và cơ bản nhất là tên đăng nhập và mật khẩu. Tương tự như trang đăng ký, trang đăng nhập cũng có form validation để xác thực mật khẩu.

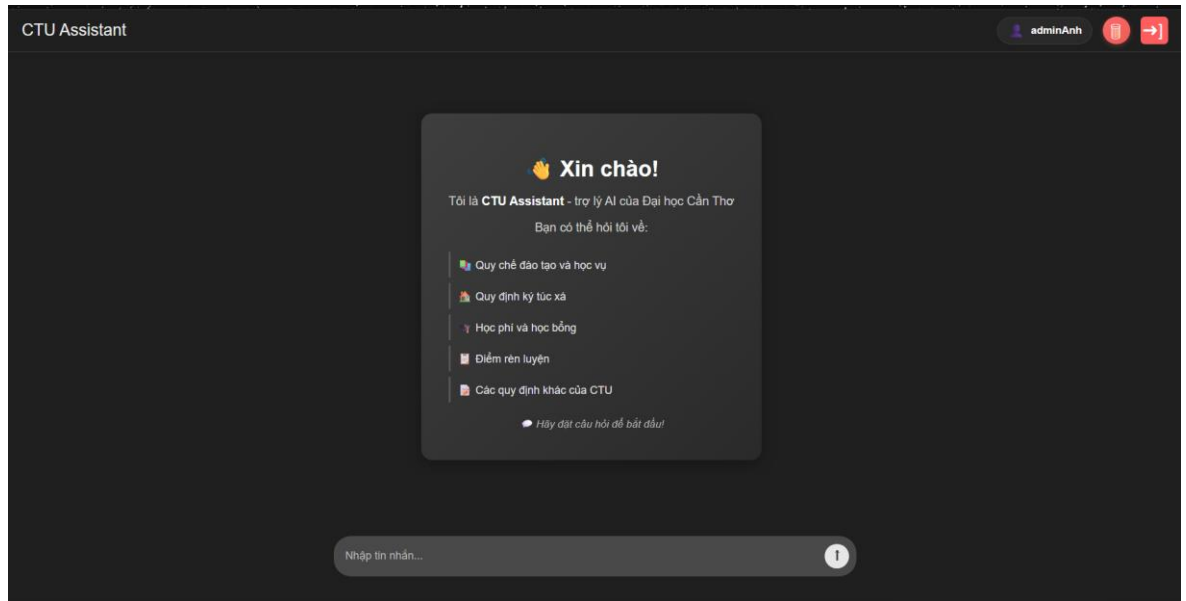


The image shows a login form titled "Đăng nhập" (Login) on a dark background. It features two input fields: "Username" and "Password". Below the password field, there is a red error message: "Login failed: Invalid username or password". A prominent green button labeled "Login" is positioned below the error message. At the bottom, there is a red link that says "Chưa có tài khoản? Đăng ký" (Don't have an account? Register).

Hình 3-10 Giao diện trang đăng nhập

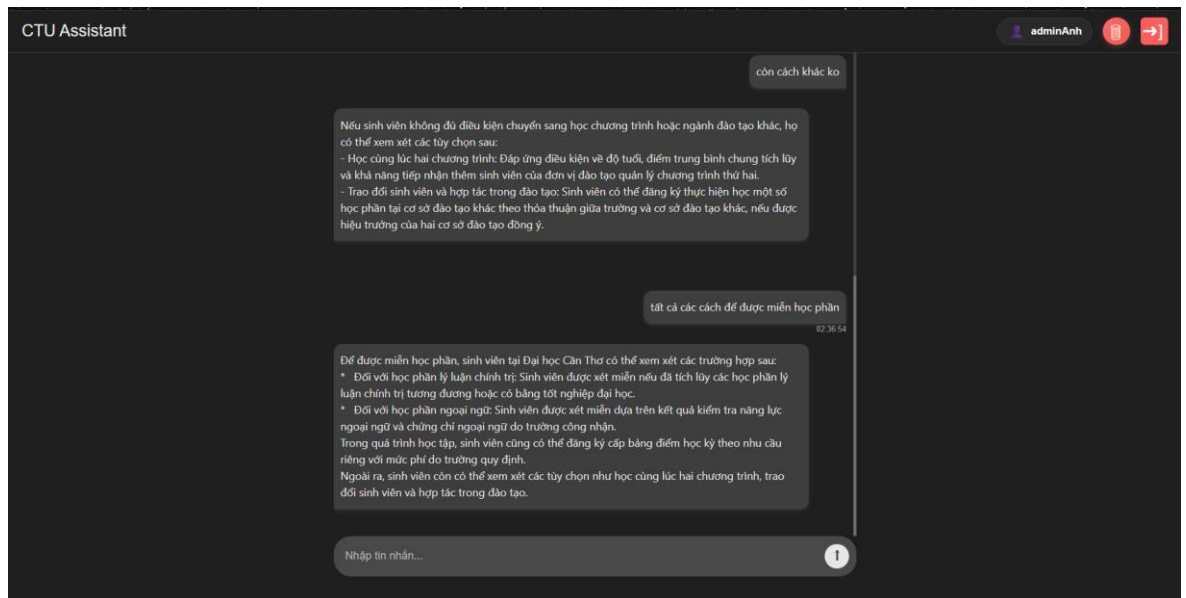
c. Trang Chat

Trang chủ của hệ thống, bao gồm thanh navbar và hộp chat, nơi mà người dùng sẽ thực hiện gửi câu hỏi của mình đến chatbot. Khi chưa có lịch sử hội thoại, trang chủ sẽ hiện thông tin lời chào đến người dùng:



Hình 3-11 Giao diện trang chủ người dùng khi chưa có đoạn chat

Giao diện hệ thống khi có lịch sử hội thoại:



Hình 3-12 Giao diện trang chủ khi đã có lịch sử chat

Hiện tại dự án chưa có chức năng đã chuỗi hội thoại, chỉ có một chuỗi duy nhất, tuy nhiên có thể mở rộng sau.

Trang chủ có các thành phần chính:

- Thanh navbar cho phép người dùng xóa lịch sử hội thoại và đăng xuất.
- Hộp chat có khả năng auto scroll (tự động cuộn) khi có tin nhắn mới gây tràn màn hình, khi người dùng di chuột vào tin nhắn sẽ tự động hiển thị thời gian gửi tin nhắn, ngoài ra ô input còn có nghe sự kiện keydown khi người dùng gõ bất kì phím nào sẽ tự động focus và ghi vào ô input.

Nhìn chung giao diện hệ thống rất đơn giản và dễ sử dụng, phần này chủ yếu tập trung vào UX giúp trải nghiệm người dùng được cải thiện đến mức tối đa.

CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG

4.1. Môi trường phát triển

4.1.1. Phần cứng

Hệ thống được phát triển trong điều kiện khá hạn chế như sau:

- CPU: Intel core i5 11400H 2.70GHz, 12 threads
- RAM: 24GB DDR4
- GPU: NVIDIA RTX 3050 (4GB VRAM)
- Ổ cứng: 512GB SSD

4.1.2. Phần mềm

- Hệ điều hành: Windows 11 Pro
- Python: 3.13.2
- IDE: Visual Studio Code
- Database: MySQL 8.0

4.1.3. Các thư viện chính

Dự án sử dụng chủ yếu các thư viện chính như sau:

- FastAPI 0.104.1 (Backend framework)
- LangChain 0.1.0 (RAG pipeline)
- FAISS-GPU 1.7.4 (Vector search)
- Sentence-Transformers 2.2.2 (Embeddings)
- rank-bm25 0.2.2 (Keyword search)
- mysql-connector-python 8.2.0

4.2. Chuẩn bị dữ liệu

4.2.1. Giai đoạn thu thập văn bản

Các tài liệu văn bản về nội quy, quy định toàn bộ được thu thập từ các trang website của Đại học Cần Thơ.

- Các dữ liệu gốc là file Word, PDF được chuyển sang định dạng .txt
- Tổng dung lượng khoảng 2.5MB text thuần

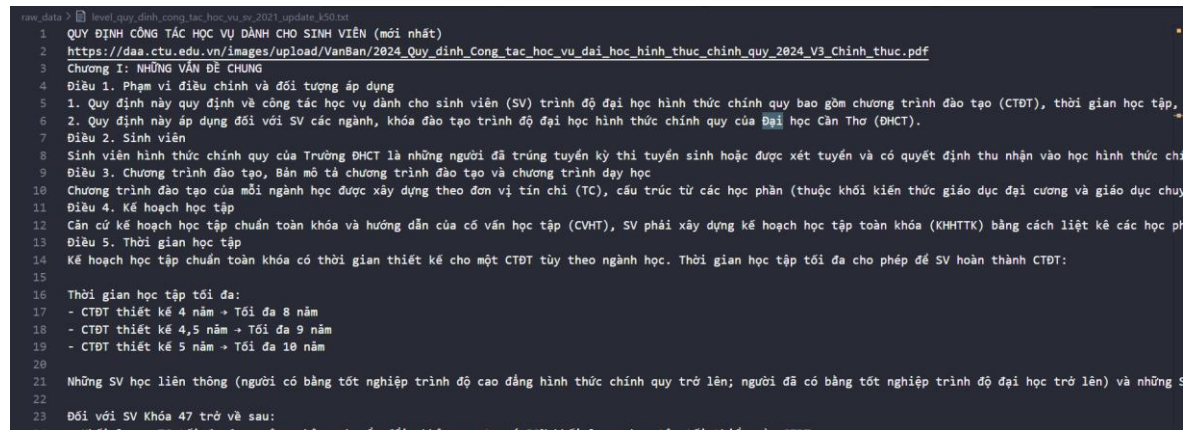
4.2.2. Giai đoạn tiền xử lý văn bản

Giai đoạn này có thể nói là quan trọng gần như bậc nhất trong hệ thống vì để có thể xây dựng và lưu trữ hiệu quả để sử dụng thì format toàn bộ các tài liệu thành một dạng thống nhất là cực kì cần thiết. Toàn bộ giai đoạn này được thực hiện bằng bằng script trong file chuanhoa.py.

Gồm các công đoạn như:

- Chuyển các nội dung dạng bảng thành văn bản thuần.
- Loại bỏ header, footer, số trang, các mục không cần thiết.
- Dòng 1 là tên văn bản, dòng 2 là liên kết gốc của văn bản.
- Tách thành cấu trúc Document→Chapter→Article→clause

Dưới đây là một đoạn văn bản sau khi chuẩn hóa:

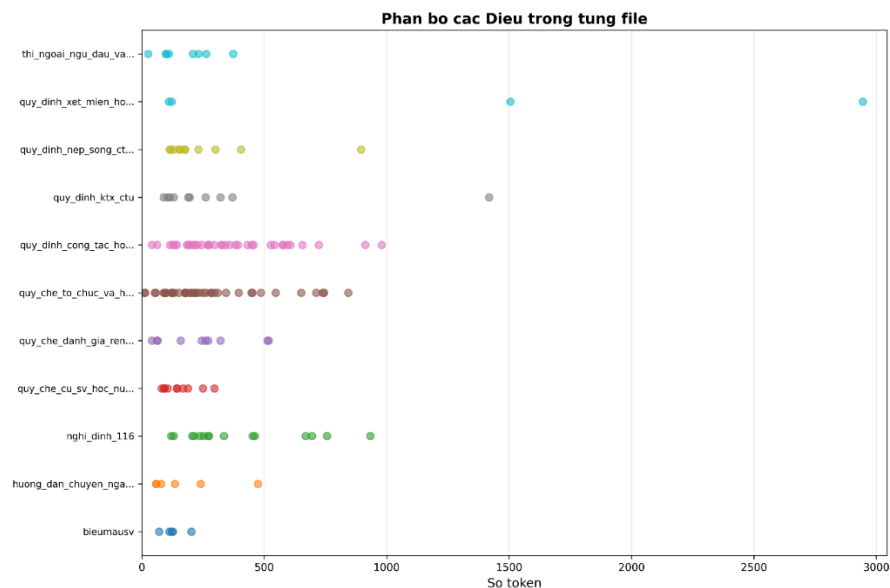


```
raw_data > level_quy_dinh_cong_tac_hoc_vu_sv_2021_update_K50.txt
1  QUY ĐỊNH CÔNG TÁC HỌC VỤ DÀNH CHO SINH VIÊN (mới nhất)
2  https://daa.ctu.edu.vn/images/upload/VanSan/2024_Quy_dinh_Cong_tac_hoc_vu_dai_hoc_hinh_thuc_chinh_quy_2024_V3_Chinh_thuc.pdf
3  Chương I: NHỮNG VẤN ĐỀ CHUNG
4  Điều 1. Phạm vi điều chỉnh và đối tượng áp dụng
5  1. Quy định này quy định về công tác học vụ dành cho sinh viên (SV) trình độ đại học hình thức chính quy bao gồm chương trình đào tạo (CTĐT), thời gian học tập,
6  2. Quy định này áp dụng đối với SV các ngành, khóa đào tạo trình độ đại học hình thức chính quy của Đại học Cần Thơ (DHCT).
7  Điều 2. Sinh viên
8  Sinh viên hình thức chính quy của Trường DHCT là những người đã trúng tuyển kỳ thi tuyển sinh hoặc được xét tuyển và có quyết định thu nhận vào học hình thức ch
9  Điều 3. Chương trình đào tạo, Bản mô tả chương trình đào tạo và chương trình dạy học
10 Chương trình đào tạo của mỗi ngành học được xây dựng theo đơn vị tín chỉ (TC), cấu trúc từ các học phần (thuộc khối kiến thức giáo dục đại cương và giáo dục chuy
11 Điều 4. Kế hoạch học tập
12 Căn cứ kế hoạch học tập chuẩn toàn khóa và hướng dẫn của cố vấn học tập (CVHT), SV phải xây dựng kế hoạch học tập toàn khóa (KHMTK) bằng cách liệt kê các học ph
13 Điều 5. Thời gian học tập
14 Kế hoạch học tập chuẩn toàn khóa có thời gian thiết kế cho một CTĐT tùy theo ngành học. Thời gian học tập tối đa cho phép để SV hoàn thành CTĐT:
15
16 Thời gian học tập tối đa:
17 - CTĐT thiết kế 4 năm → Tối đa 8 năm
18 - CTĐT thiết kế 4,5 năm → Tối đa 9 năm
19 - CTĐT thiết kế 5 năm → Tối đa 10 năm
20
21 Những SV học liên thông (người có bằng tốt nghiệp trình độ cao đẳng hình thức chính quy trở lên; người đã có bằng tốt nghiệp trình độ đại học trở lên) và những S
22
23 Đối với SV Khóa 47 trở về sau:
```

Hình 4-1 Một đoạn văn bản sau khi chuẩn hóa

4.2.3. Chunking

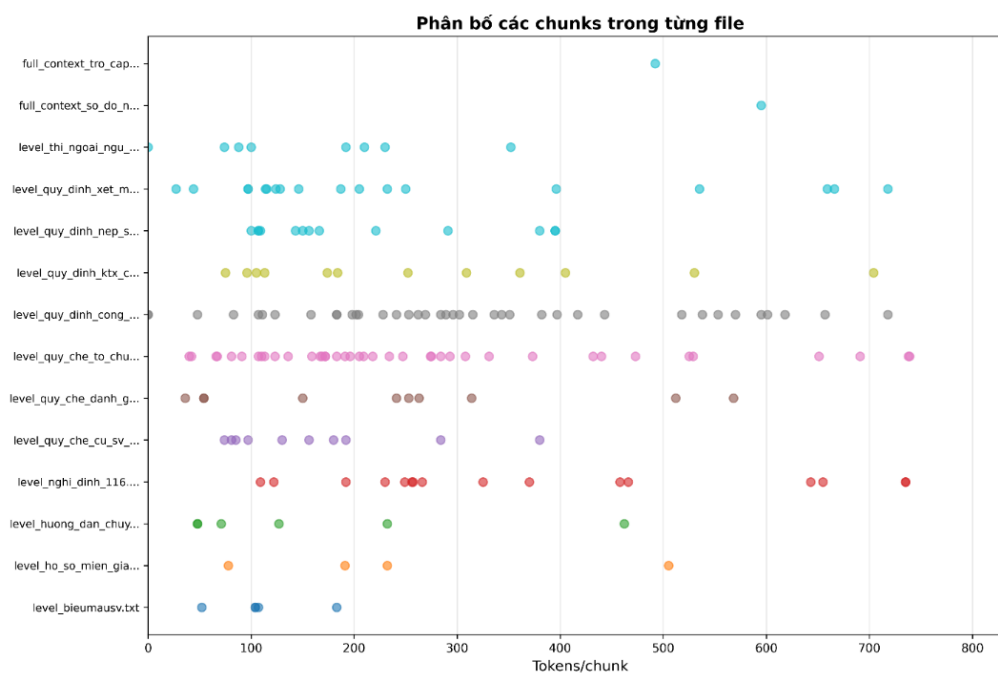
Tổng quan trạng thái các chunk theo tokens của từng file trước khi cắt lại với các tham số hợp lý, với mỗi toàn bộ điều là một chunk:



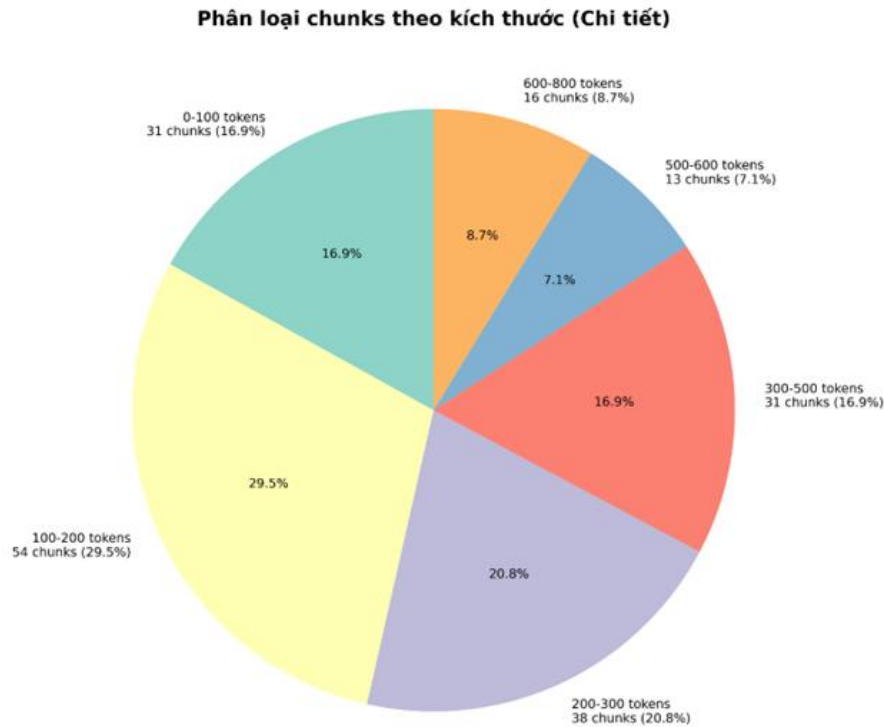
Hình 4-2 Sơ đồ phân bố chunk theo từng điều trước khi cắt hợp lý

Ta có thể thấy rõ, các chunk phân bố rất không đồng đều sẽ làm ảnh hưởng quá trình tìm kiếm.

Hình 4-3 mô tả sự phân bố các chunks theo tokens (min 0, max 739) ở mỗi documents:



Hình 4-3 Sự phân bố chunks theo từng file



Hình 4-4 Biểu đồ phân loại chunks theo kích thước

Dựa vào Hình 4-4 ta thấy sau khi phân cắt hợp lý, các chunk phân bố khá ổn và không còn lệch nhiều như trước:

Có 50% chunks $\leq$ 218 tokens,

80% chunks $\leq$ 426 tokens và

95% chunks $\leq$ 659 tokens

Vì dữ liệu thực bao gồm các “điều” rất ngắn và rất dài, không thể có thể phân bố đều tokens ở một mức nào đó, nên chuẩn hóa các token ở mức trung bình 268 tokens/chunk là khá ổn và hợp lý cho mô hình.

Sau nhiều lần thực nghiệm, đây là các thông số tốt nhất cho chunking của dự án

- **Chiến lược:** RecursiveCharacterTextSplitter từ LangChain
- **Chunk size:** 750 tokens (ước tính khoảng 577 từ tiếng Việt)
- **Overlap:** 120 tokens (giữ context giữa các chunks)
- **Separators:** Ưu tiên theo \nĐiều, \nNhóm, \n1., \na), \n\n.

```
# Separators ưu tiên cấu trúc văn bản pháp lý Việt Nam:
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=CHUNK_SIZE,
    chunk_overlap=CHUNK_OVERLAP,
    length_function=token_length_function,
    separators=[
        "\nĐiều ",
        "\nNhóm ",
        "\n1. ", "\n2. ", "\n3. ", "\n4. ", "\n5. ", "\n6. ", "\n7. ", "\n8. ", "\n9. ", "\n10. ",
        "\na) ", "\nb) ", "\nc) ", "\nd) ", "\ne) ", "\nf) ",
        "\n\n",
        "\n"
    ]
)
```

Hình 4-5 Đoạn chương trình về độ ưu tiên khi chunking

4.2.4. Import vào MySQL

Sau khi đã chia chunk, tiến hành lưu vào MySQL theo logic đã thiết kế ở giai đoạn thiết kế cơ sở dữ liệu.

4.3. Xây dựng hệ thống tìm kiếm

4.3.1. Tạo FAISS Semantic Index

Quá trình xây dựng được thực hiện tại file: backend/app/services/faiss_builder.py

Quy trình:

1. Load model embeddings: bkai-foundation-models/vietnamese-bi-encoder
2. Lấy toàn bộ clauses từ MySQL (sử dụng GROUP_CONCAT để lấy full context)
3. Tải xử lý văn bản theo turn FAISS
4. Embedding từng clause → vector 768 chiều
5. Tạo 2 index:
 - index.faiss: từ text có dấu
 - index_none_accent.faiss: từ text không dấu
6. Lưu metadata: id_mapping.json

Trong quá trình phát triển, hệ thống từng sử dụng model all-MiniLM-L6-v2 384 chiều để embedding các văn bản. Tuy nhiên, nhận thấy model này không hỗ trợ tốt tiếng Việt cũng như không hiệu quả cho dự án, tôi quyết định dùng bkai-foundation-models để thay thế vì nó hỗ trợ tốt tiếng Việt và phù hợp cho dự án.

4.3.2. Xây dựng BM25 Index

Quá trình xây dựng được thực hiện tại file: backend/app/services/bm25_index.py

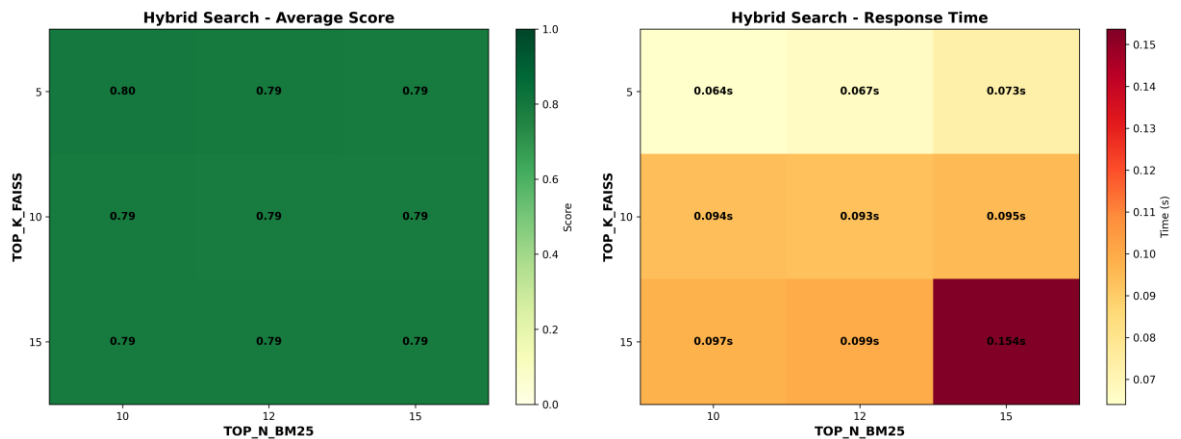
Quy trình:

1. Load clauses từ database
2. Tokenize, tái chuẩn hóa văn bản: tách từ + loại stopwords (vie_stopwords.txt)

3. Tạo BM25 index (rank-bm25 library)
4. Rebuild mỗi lần khởi động backend

Quá trình xây dựng BM25 Index khá nhẹ nhàng vì thư viện hỗ trợ dường như tất cả, các chỉ số không cần phải tính thủ công mà chỉ cần chuẩn hóa dữ liệu lại “một chút” là có thể sử dụng tốt.

4.3.3. Xây dựng tham số tối ưu cho tìm kiếm



Hình 4-6 So sánh độ chính xác theo từng tham số

Dựa vào hình 4-6, ta thấy ở top_k=5 và top_n=10 cho độ chính xác lẫn thời gian phản hồi nhanh nhất. Tuy nhiên, trong một số điều kiện top k quá thấp sẽ dẫn đến thiếu ngữ cảnh nên tôi quyết định thử nghiệm lần lượt và cuối cùng rút ra tham số tối ưu nhất:

- Top_k = 12
- Top_N=10

4.4. Xây dựng backend (FastAPI)

4.4.1. Cấu trúc dự án

Dưới đây là cấu trúc chính của backend:

```
backend/app/
├── main.py           # Entry point
├── db.py             # MySQL connection pool
├── routers/
│   ├── ask.py        # POST /api/ask
│   └── user.py        # /api/user
├── services/
│   ├── hybrid.py
│   ├── search_services.py
│   ├── faiss_builder.py
│   ├── bm25_index.py
│   └── utils.py
└── scripts/
    └── embedding.py
```

```
└─ build_ollama.py
```

4.4.2. Triển khai RAG pipeline

Được thực hiện trong file /routers/ask.py

Luồng xử lý:

1. Nhận POST request: {"question": "...", "user_id": 3}
2. Query preprocessing: chuẩn hóa, mở rộng từ đồng nghĩa
3. Hybrid Search: gọi BM25 + FAISS → RRF
4. Build prompt: System prompt + context + history (20 messages) + question
5. LLM generation: Gọi Ollama qua LangChain
6. Streaming: Yield từng token qua SSE
7. Save to DB: Lưu question + answer vào messages

Luồng xử lý trên đã thực hiện chính xác theo những gì đã thiết kế ở phần trước đó

4.4.3. Tích hợp LLM (Ollama)

Model: Llama3.1 (8B parameters)

Cài đặt:

```
from langchain_ollama.llms import OllamaLLM
_model = OllamaLLM(model=model_name,
                    options={
                        "temperature": 0.3,
                        "top_p": 0.5,
                        "top_k": 30,
                        "num_predict": 1024, # Tăng lên để không bị cắt câu trả lời
                        "num_ctx": 8192,      # Context window
                        "num_thread": 8,      # Sử dụng 8 CPU threads
                        "num_gpu": 1,         # Dùng GPU nếu có (NVIDIA)
                        "repeat_penalty": 1.1, # Tránh lặp từ
                    })
```

4.5. Xây dựng Frontend

4.5.1. Công nghệ

Dự án sử dụng cách viết web thuần nhất, không sử dụng framework:

- HTML5 + CSS3 (Vanilla, không framework)
- JavaScript ES6 (Class-based)
- Fetch API + EventSource (SSE)

4.5.2. Cấu trúc thư mục

```
frontend/
```

```
| index.html      # Trang chính
| login.html     # Đăng nhập
| register.html  # Đăng ký
| css/
| | auth.js
| | style.js
| | chat.css
| js/
| | auth.js      # Login/Register
| | chat.js      # Chat logic
| | config.js    # Cấu hình
| | main.js      # Xử lý chính
| | utils.js     # Helpers
```

4.5.3. Xử lý streaming

Code xử lý (chat.js):

```
1  class Chat {
90  async chatStream() {
123      // Backend trả về StreamingResponse, đọc từng chunk
124      const reader = res.body.getReader();
125      const decoder = new TextDecoder();
126      botReply = "";
127      botBubble.innerHTML = "";
128
129      while (true) {
130          const { done, value } = await reader.read();
131          if (done) break;
132
133          const chunk = decoder.decode(value, { stream: true });
134          botReply += chunk;
135          botBubble.innerHTML = Utils.formatMessage(botReply);
136          Utils.scrollToBottom(this.chatbox);
137      }
138  } catch (err) {
139      console.error("Chat error:", err);
140      botReply = "Đã có lỗi xảy ra, vui lòng thử lại sau.";
141      botBubble.innerHTML = `<em>${botReply}</em>`;
142  }
```

Hình 4-7 Xử lý hiển thị streaming response

Hình 4-6 là đoạn chương trình thuộc phương thức chatStream lớp Chat được cài đặt để có thể hiển thị và decode các tokens theo từng chunks trả về từ backend ra frontend. User sẽ thấy câu trả lời từ trái sang phải.

4.6. Kiểm thử hệ thống

4.6.1. Kiểm thử chức năng

Do thời gian và phạm vi đề tài hạn chế, hệ thống được kiểm thử thủ công thông qua giao diện web:

Kiểm thử tìm kiếm cơ bản: Nhập các câu hỏi đơn giản về quy chế (ví dụ: "điều kiện tốt nghiệp", "cách tính điểm trung bình")

Kiểm thử xử lý tiếng Việt:

- Câu hỏi có dấu/không dấu
- Câu hỏi có từ đồng nghĩa
- Câu hỏi có stopwords

Kiểm thử streaming: Quan sát phản hồi hiển thị từng phần, không bị đứng hay lỗi.

Kiểm thử lưu trữ: Kiểm tra dữ liệu conversation và message được lưu đúng vào MySQL.

4.6.2. Kiểm thử đánh giá

Để đánh giá chất lượng hệ thống một cách khách quan, tôi đã:

1. Chuẩn bị bộ câu hỏi: Tạo 50 câu hỏi thực tế từ 14 văn bản quy chế CTU, bao gồm:

- Câu hỏi về điều kiện (tốt nghiệp, miễn giảm học phí, xét học bổng...)
- Câu hỏi về quy trình (đăng ký học phần, khiếu nại điểm...)
- Câu hỏi về thông tin (phí ký túc xá, quy định nội trú...)
- Cả câu hỏi ngắn và dài

2. Đánh giá kết quả: So sánh câu trả lời của hệ thống với câu trả lời chuẩn (ground truth):

- **Exact Match:** Câu trả lời khớp hoàn toàn ($\text{similarity} \geq 0.9$)
- **Partial Match(overlap):** Câu trả lời có chồng lặp một phần ($0.5 \leq \text{similarity} < 0.9$)
- **No Match:** Câu trả lời sai hoặc không liên quan

3. Kết quả đạt được:

- Tổng số câu hỏi: **50**
- Trả lời đúng: **42 câu (84%)**
 - Exact Match: 38 câu (76%)
 - Partial Match: 4 câu (8%)
- Trả lời sai: 8 câu (16%)
- Độ tương đồng trung bình: **81.2%**

- Thời gian phản hồi trung bình: **33.41 giây** (dao động 12-77 giây)

4.6.3. Kiểm thử phi chức năng

Hiệu năng: Thời gian phản hồi trung bình ~33 giây (phụ thuộc vào độ phức tạp câu hỏi và độ dài context)

Tính ổn định: Hệ thống chạy liên tục không bị crash trong quá trình test 50 câu hỏi

Khả năng mở rộng: Có thể thêm văn bản mới bằng cách chạy lại script build_index.py

4.7. Triển khai (Deployment)

4.7.1. Khởi động Backend

Ở file main.py, khi khởi động backend ở endpoint root (/) phương thức GET, sẽ tự động trả về giao diện frontend thông qua phương thức FileResponse.

```
cd backend
python -m uvicorn app.main:app --reload --port 8000
```

4.7.2. Khởi động Frontend

Mở live server để khởi động và triển khai Frontend

4.7.3. Khởi động Ollama

```
ollama run llama3.1:8b
```

4.7.4. Kiểm tra hoạt động

Kiểm tra thủ công bằng cách mở đường dẫn: <http://localhost:8000/>

4.7.5. Test API

Tiến hành kiểm tra các endpoint API bằng Postman:

Test API /api/ask (Gửi câu hỏi):

- **Method:** POST
- **URL:** http://localhost:8000/ask
- **Headers:** Content-Type: application/json
- **Body:**

```
{
  "question": "Điều kiện tốt nghiệp của sinh viên là gì?",
  "user_id": 1,
  "conversation_id": 1
}
```

- **Kết quả mong đợi:**
 - Status 200
 - Response trả về data

- Data chứa câu trả lời từ LLM và context sources

Test các API dành cho user khác (/api/user/)

Kết quả kiểm thử Postman:

- Tất cả endpoint hoạt động đúng
- Response time API ask: ~30-40s (do LLM generation)
- Không có lỗi 500 (Internal Server Error)

CHƯƠNG 5. ĐÁNH GIÁ VÀ KẾT QUẢ

5.1. Phương pháp đánh giá

5.1.1. Bộ dữ liệu đánh giá

Bộ dữ liệu đánh giá được trích xuất từ 14 văn bản quy chế chính thức của Đại học Cần Thơ:

- **Số lượng chunks:** ~183 chunks
- **Bộ câu hỏi test:** 50 câu hỏi thực tế bao gồm:
 - Câu hỏi về điều kiện (15 câu): Điều kiện tốt nghiệp, miễn giảm học phí, xét học bổng...
 - Câu hỏi về quy trình (20 câu): Đăng ký học phần, khiếu nại điểm, chuyển ngành...
 - Câu hỏi về thông tin (15 câu): Phí ký túc xá, quy định nội trú, học phí...
- **Câu trả lời chuẩn (ground truth):** Được trích xuất trực tiếp từ văn bản quy chế gốc

5.1.2. Tiêu chí đánh giá

Để có thể đánh giá gần chính xác nhất, đề tài thực hiện loạt các phương pháp đánh giá sau:

a) Accuracy (Độ chính xác)

- Đo lường tự động và thủ công tỷ lệ câu trả lời đúng so với tổng số câu hỏi
- Phân loại kết quả:
 - **Exact Match** (Khớp hoàn toàn): $\text{Similarity} \geq 0.9$
 - **Partial Match** (Khớp một phần): $0.5 \leq \text{Similarity} < 0.9$
 - **No Match** (Không khớp): $\text{Similarity} < 0.5$

Công thức được sử dụng đánh giá mô hình:

$$\text{Accuracy}(\%) = \frac{1}{N} \sum_{i=1}^N \text{sim}_i \cdot 100$$

Trong đó:

- N là tổng số câu hỏi

- sim_i là độ chính xác cũng như điểm của câu hỏi i

b) Similarity Score (Độ tương đồng ngữ nghĩa)

- Sử dụng mô hình **keepitreal/vietnamese-sbert** để tính cosine similarity
- So sánh embedding của câu trả lời hệ thống với câu trả lời chuẩn
- Điểm từ 0-1 (càng cao càng giống)
- Được đánh giá tự động

c) Response Time (Thời gian phản hồi)

- Đo thời gian từ lúc gửi request đến khi nhận đủ câu trả lời
- Bao gồm: Thời gian tìm kiếm + Thời gian LLM generate

5.1.3. Quy trình đánh giá

1. Gửi từng câu hỏi trong bộ 50 câu qua API /ask
2. Thu thập câu trả lời của hệ thống
3. Tính similarity score giữa câu trả lời và ground truth
4. Phân loại kết quả (Exact/Partial/No Match)
5. Ghi nhận thời gian phản hồi
6. Lưu kết quả vào file JSON để phân tích
7. Tiến hành tái đánh giá thủ công bằng việc kiểm tra câu trả lời từng câu hỏi và tinh chỉnh lại similarity và các thông số khác
8. Tiến hành viết script tạo biểu đồ trực quan dựa trên kết quả cuối cùng từ file json

5.2. Kết quả đánh giá

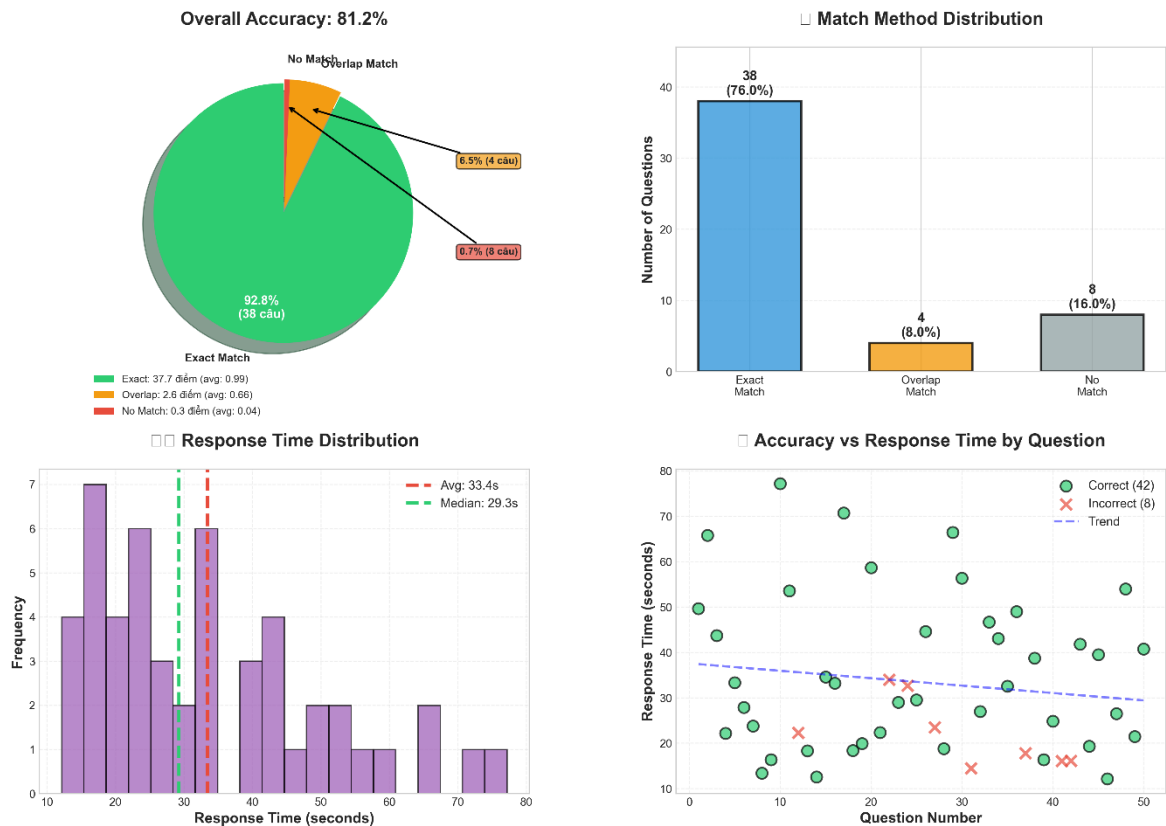
5.2.1. Kết quả tổng quan

Bảng 5-1 Bảng kết quả tổng quan

Chỉ số	Giá trị
Tổng số câu hỏi	50
Câu trả lời đúng	42 (84%)
- Exact Match	38 (76%)
- Partial Match	4 (8%)
Câu trả lời sai	8 (16%)
Độ tương đồng trung bình	81.2%
Thời gian phản hồi TB	33.41 giây
Thời gian nhanh nhất	12.13 giây
Thời gian chậm nhất	77.17 giây

5.2.2. Phân tích chi tiết

LLM Answer Quality Benchmark Analysis



Hình 5-1 Kết quả đạt được của dự án

Hình 5-1 là 4 biểu đồ thể hiện hiệu năng của hệ thống, lần lượt là:

Overall Accuracy: được tính dựa trên tỉ số của số điểm đạt được trong quá trình test và tổng số điểm. Hệ thống được theo cách này vì đây là cách tương đối chính xác khi đánh giá một mô hình chatbot, bởi lẽ chatbot thường trả lời theo nhiều cách khác nhau nên chỉ cần so sánh câu trả lời có đầy đủ thông tin hay không, hay thiếu một phần thông tin mà cho điểm của từng câu hỏi khác nhau.

- **Exact Match (76%):** Hệ thống trả lời chính xác, trích dẫn đúng điều khoản
 - Ví dụ: "Điều kiện tốt nghiệp cần GPA ≥ 2.0 " → Similarity 0.9
- **Overlap (8%):** Hệ thống trả lời đúng ý nhưng thiếu chi tiết hoặc diễn đạt khác
 - Ví dụ: Câu hỏi về "quy trình khiếu nại điểm" → Trả lời đúng nhưng thiếu deadline
- **No Match (16%):** Hệ thống trả lời sai hoặc không liên quan
 - Nguyên nhân: Câu hỏi mơ hồ, thông tin không có trong văn bản, hoặc LLM hallucination

Match Method Distribution: Biểu đồ cột biểu hiện kết quả của câu trả lời.

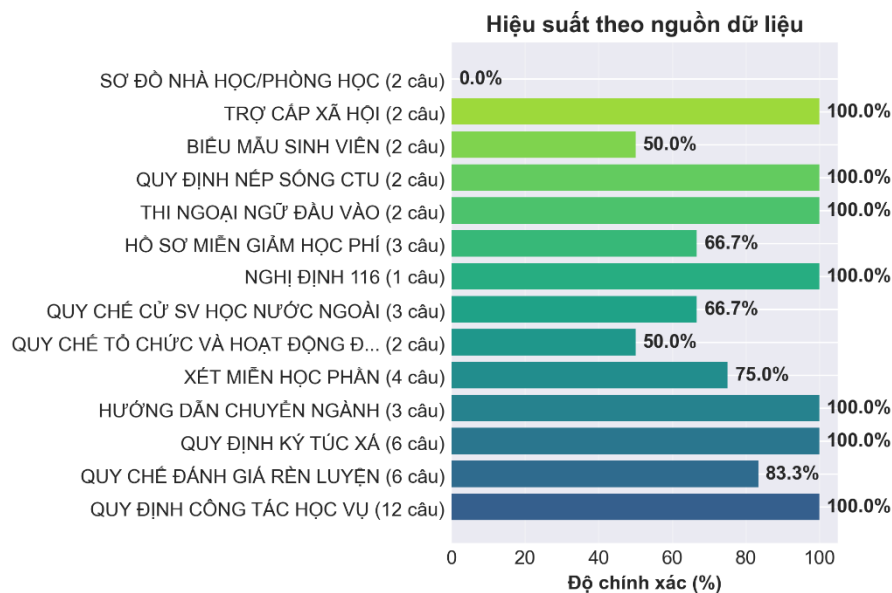
Reponse Time Distribution: Biểu đồ thể hiện tương quan giữa số lượng câu hỏi và thời gian phản hồi câu hỏi (tính bằng giây).

Các yếu tố ảnh hưởng đến thời gian phản hồi:

- Độ dài context được truy xuất (nhiều chunks → chậm hơn)
- Độ phức tạp câu hỏi → LLM cần nhiều thời gian suy luận
- Không có GPU mạnh → Tốc độ inference chậm

Accuracy Vs Reponse Time by Question: biểu đồ tương quan thời gian đáp ứng với từng câu hỏi và sự đúng sai. Qua biểu đồ ta thấy các câu trả lời chưa chính xác phần lớn có thời gian đáp ứng nhanh, các câu trả lời chậm thì lại hoàn toàn chính xác. Điều này phản ánh đáng kể hiệu suất của mô hình LLM được lựa chọn.

Việc đánh giá mô hình dựa hiệu suất nguồn dữ liệu cũng rất quan trọng:



Hình 5-2 Hiệu suất dựa vào nguồn dữ liệu

Ta thấy được ở file sơ đồ nhà học/phòng học thì hiệu suất bằng 0, chứng tỏ mô hình không hiểu được quy tắc quy đổi và so sánh tương đương các ký hiệu. Việc này có thể cải thiện bằng cách fine-tuning để dạy cho mô hình predict (dự đoán) được phòng nào ở vị trí nào.

5.3. So sánh với các giải pháp khác

Bảng 5-2 Bảng so sánh với các giải pháp khác

Tiêu chí	Hệ thống đề xuất	ChatGPT/Gemini (public)	Rule-based Chatbot
Accuracy	81.2%	~90% (nếu có context)	~60%

Response time	~33s	~5-10s	<1s
Bảo mật dữ liệu	Chạy local	Cloud (rủi ro rò rỉ)	Local
Chi phí	Miễn phí	Tính phí API	Miễn phí
Tùy chỉnh	Dễ dàng	Hạn chế	Dễ dàng
Xử lý tiếng Việt	Tốt	Tốt	Phụ thuộc template

Tốc độ phản hồi của hệ thống phụ thuộc hoàn toàn vào GPU hoặc CPU.

Tùy theo nhu cầu và mục đích sử dụng, người dùng có thể lựa chọn phương pháp hợp lý phù hợp với mình.

5.4. Phân tích ưu điểm

1. **Độ chính xác cao:** 81.2% accuracy, đáp ứng tốt yêu cầu thực tế
2. **Bảo mật dữ liệu:** Không gửi dữ liệu nhạy cảm lên cloud
3. **Tiết kiệm chi phí:** Không cần trả phí API cho ChatGPT/Gemini
4. **Kiểm soát hoàn toàn:** Có thể tinh chỉnh mô hình, prompt, thuật toán search
5. **Xử lý tiếng Việt tốt:**
 1. Loại bỏ stopwords tiếng Việt
 2. Xử lý câu có dấu/không dấu
 3. Từ điển từ đồng nghĩa
6. **Hybrid Search hiệu quả:** Kết hợp BM25 (keyword) + FAISS (semantic) → Tăng độ phủ kết quả

5.5. Phân tích hạn chế

5.5.1. Về hiệu năng

Thời gian phản hồi chậm: Trung bình 33 giây (do chạy LLM local trên GPU hiệu năng thấp)

5.5.2. Về độ chính xác

8 câu trả lời sai (16%):

- 3 câu: Câu hỏi mơ hồ, thiếu context
- 2 câu: LLM hallucination (tự suy diễn sai)
- 3 câu: Thông tin không có trong văn bản

5.5.3. Về dữ liệu

- **Giới hạn 14 văn bản:** Chưa bao phủ hết các quy định của trường
- **Chưa có cơ chế update tự động:** Khi quy chế thay đổi cần rebuild thủ công

5.5.4. Về giao diện

- **Chưa có mobile app:** Sinh viên phải truy cập qua web browser
- **Thiếu tính năng:** Bookmark câu trả lời, voice input, voice output, multi-conversations

PHẦN KẾT

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận

6.1.1. Tổng kết công việc đã thực hiện

Niên luận đã hoàn thành mục tiêu xây dựng hệ thống chatbot tư vấn quy chế tự động cho sinh viên Trường Đại học Cần Thơ dựa trên công nghệ RAG (Retrieval-Augmented Generation) và mô hình ngôn ngữ lớn chạy local. Hệ thống cho phép sinh viên tra cứu thông tin quy chế nhanh chóng, chính xác và bảo mật mà không phụ thuộc vào các dịch vụ cloud trả phí, bao gồm:

- **Tiền xử lý dữ liệu** từ 14 văn bản quy định
- **Xây dựng Hybrid Search** (BM25 + FAISS) để truy xuất thông tin chính xác
- **RAG Pipeline** kết hợp tìm kiếm và LLM để sinh câu trả lời
- **Backend FastAPI** hỗ trợ API /search và /ask
- **Frontend Web** hiển thị kết quả theo thời gian thực (streaming)
- **Lưu trữ hội thoại** vào MySQL phục vụ theo dõi và phân tích

6.1.2. Kết quả đạt được

Hệ thống được benchmark bằng 50 câu hỏi thực tế:

- **Success Rate: 84%**
- **Exact Match: 76%**
- **Partial Match: 8%**
- **No Match: 16%**
- **Similarity trung bình (accuracy): 81.2%**
- **Avg response time: 33.41s**

Kết quả chứng minh mô hình đủ khả năng phục vụ tư vấn văn bản quy định ở mức chất lượng tốt.

Về mặt ứng dụng thực tế: Hệ thống có thể trả lời chính xác hầu hết câu hỏi thường gặp của sinh viên về:

- Điều kiện tốt nghiệp, chuyển ngành, xét học bổng
- Quy trình đăng ký học phần, khiếu nại điểm
- Thông tin ký túc xá, học phí, miễn giảm học phí

Giúp Giảm tải công việc cho phòng ban hành chính và tư vấn sinh viên và sinh viên có thể tra cứu thông tin 24/7 mà không cần chờ đợi.

6.1.3. Đóng góp của đề tài

Đề tài mang lại ba đóng góp chính:

1. Đóng góp về mặt khoa học:

- Nghiên cứu và áp dụng thành công kiến trúc RAG cho bài toán QA tiếng Việt
- So sánh hiệu quả của BM25, FAISS và hybrid search trên dữ liệu quy chế
- Đánh giá khả năng của LLM local (Llama 3.1 8B) trong việc xử lý văn bản pháp quy tiếng Việt

2. Đóng góp về mặt thực tiễn:

- Cung cấp giải pháp chatbot tư vấn quy chế cho tổ chức giáo dục với chi phí thấp
- Chứng minh tính khả thi của việc triển khai LLM local thay vì cloud API
- Xây dựng quy trình tái sử dụng được cho các trường đại học khác

3. Đóng góp về mặt kỹ thuật:

- Mã nguồn mở, tài liệu đầy đủ, dễ triển khai và tùy chỉnh
- Pipeline xử lý dữ liệu tiếng Việt có thể áp dụng cho các domain khác

6.1.4. Ý nghĩa thực tiễn

Hệ thống chatbot quy chế CTU không chỉ là một đề tài nghiên cứu mà còn có khả năng triển khai thực tế:

- Giúp sinh viên tiết kiệm thời gian tra cứu thông tin
- Giảm áp lực cho bộ phận tư vấn sinh viên
- Đảm bảo thông tin quy chế được truyền đạt chính xác, nhất quán
- Có thể mở rộng cho các đối tượng khác: giảng viên, cán bộ, phụ huynh

6.2. Hướng phát triển

6.2.1. Cải thiện tập dữ liệu

- Xây dựng pipeline tự động normalize và update các văn bản mới
- Thêm timestamp để detect văn bản cũ

6.2.2. Cải thiện độ chính xác

- Thay đổi LLM khác mạnh hơn
- Sử dụng mô hình nhiều tham số hơn
- Fine-tune LLM theo domain văn bản quy định
- Mở rộng chiến lược mở rộng truy vấn (query expansion)
- Tối ưu prompt để giảm lỗi suy luận

6.2.3. Tối ưu hiệu năng

- Sử dụng GPU mạnh để tăng tốc inference
- Áp dụng model quantization (4-bit/8-bit)
- Implement caching cho các câu hỏi phổ biến
- Thuê GPU
- Chuyển sang FAISS IVF hoặc HNSW để tăng tốc độ search
- Sử dụng caching embedding / caching response
- Tối ưu chunk size để giảm số lượng embedding

6.2.4. Mở rộng chức năng

- Multi-turn conversation nâng cao
- Tích hợp giọng nói (speech-to-text, text-to-speech)
- Tạo ứng dụng mobile (Android/iOS)
- Tích hợp web search fallback

6.2.4. Cải thiện trải nghiệm người dùng

- Gợi ý câu hỏi liên quan
- Highlight nguồn trích dẫn trong văn bản
- Cho phép người dùng đánh giá câu trả lời
- Tự động nhóm lại hội thoại theo chủ đề.

Lời kết

Niên luận "Xây dựng hệ thống chatbot tư vấn quy chế sinh viên sử dụng RAG và LLM" đã chứng minh tính khả thi của việc áp dụng công nghệ AI hiện đại vào bài toán thực tế tại Việt Nam. Với kết quả đạt được 81.2% accuracy và khả năng chạy hoàn toàn local, hệ thống mở ra hướng đi mới cho các tổ chức giáo dục nhỏ muốn triển khai chatbot thông minh mà không phụ thuộc vào các dịch vụ cloud đắt đỏ.

Mặc dù còn nhiều hạn chế cần khắc phục, đặc biệt về thời gian phản hồi và độ chính xác, nhưng hệ thống đã đặt nền móng vững chắc cho những cải tiến trong tương lai. Với hướng phát triển rõ ràng, em tin rằng hệ thống có thể trở thành công cụ hữu ích không chỉ cho sinh viên CTU mà còn cho nhiều cơ sở giáo dục Việt Nam nói chung.

Em xin chân thành cảm ơn sự hướng dẫn tận tình của thầy Ts. Trần Việt Châu trong suốt quá trình thực hiện niên luận này. Đây là trải nghiệm quý báu giúp em hiểu sâu hơn về AI, NLP và cách áp dụng công nghệ để giải quyết vấn đề thực tế.

TÀI LIỆU THAM KHẢO

Tiếng Anh

- [1] Lewis, P., Perez, E., Piktus, A., et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 9459-9474.
- [2] Reimers, N., & Gurevych, I. (2019). "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks". *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 3982-3992.
- [3] Cormack, G.V., Clarke, C.L.A., & Buettcher, S. (2009). "Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods". *Proceedings of the 32nd ACM SIGIR*, 758-759.
- [4] Zhu, Y., Yuan, H., Wang, S., et al. (2023). "Large Language Model Based Long-tail Query Rewriting in Taobao Search". *arXiv preprint arXiv:2311.03758*.

Tiếng Việt

- [5] Nguyễn Văn Vinh, Trần Thị Oanh, & Lê Anh Cường (2022). "Ứng dụng mô hình BERT cho bài toán phân loại văn bản tiếng Việt". *Tạp chí Khoa học và Công nghệ*, 60(5), 123-135.
- [6] Trần Minh Tuấn, & Phạm Quang Nhật Minh (2023). "Xây dựng hệ thống hỏi đáp tự động tiếng Việt sử dụng kỹ thuật Retrieval-Augmented Generation". *Hội nghị Khoa học Công nghệ Quốc gia lần thứ XVI*, 45-52.
- [7] Đỗ Trung Tuấn, & Nguyễn Cam Tú (2021). "Nghiên cứu các phương pháp embedding cho văn bản tiếng Việt". *Tạp chí Tin học và Điều khiển học*, 37(3), 267-280.
- [8] Võ Văn Hải, & Lê Thị Mai Anh (2023). "Chatbot hỗ trợ sinh viên tra cứu thông tin sử dụng Deep Learning". *Kỷ yếu Hội nghị Khoa học trẻ toàn quốc*, 156-163.

Tài liệu kỹ thuật và Documentation

- [9] Meta AI (2024). "Llama Models Documentation". Truy cập từ: <https://ai.meta.com/llama/>
- [10] Ollama (2024). "Ollama Documentation - Run Large Language Models Locally". Truy cập từ: <https://ollama.ai/>
- [11] Facebook AI Research (2024). "FAISS: A Library for Efficient Similarity Search". Truy cập từ: <https://github.com/facebookresearch/faiss>
- [12] Hugging Face (2024). "Transformers Documentation". Truy cập từ: <https://huggingface.co/docs/transformers/>

[13] FastAPI (2024). "FastAPI Documentation". Truy cập từ: <https://fastapi.tiangolo.com/>

[14] Sentence Transformers (2024). "Sentence-BERT Documentation". Truy cập từ: <https://www.sbert.net/>

Dataset và Pre-trained Models

[15] Keepitreal (2024). "Vietnamese Sentence BERT Model". Hugging Face. Truy cập từ: <https://huggingface.co/keepitreal/vietnamese-sbert>

[16] VnCoreNLP (2023). "A Vietnamese Natural Language Processing Toolkit". Truy cập từ: <https://github.com/vncorenlp/VnCoreNLP>

[17] UIT-VNUHCM (2023). "Vietnamese Stopwords List". Truy cập từ: <https://github.com/stopwords/vietnamese-stopwords>

Quy chế và Văn bản pháp lý CTU

[18] Trường Đại học Cần Thơ (2021). "Quy định công tác học vụ sinh viên hệ chính quy (Ban hành kèm theo Quyết định số 2071/QĐ-ĐHCT)".

[19] Trường Đại học Cần Thơ (2022). "Quy chế đánh giá kết quả rèn luyện sinh viên".

[20] Trường Đại học Cần Thơ (2023). "Quy định về ký túc xá sinh viên".

[21] Trường Đại học Cần Thơ (2024). "Quy chế tổ chức và hoạt động của Trường Đại học Cần Thơ".

PHỤ LỤC

Hướng dẫn cài đặt và source code dự án

Toàn bộ mã nguồn, hướng dẫn và mọi thứ cần thiết được public tại github:

<https://github.com/anh1650000/chatbot-ctu-regulations>